Overview

🎯 In this course, you will explore how to maintain synchronization between a test script and the Application Under Test (AUT) when using Squish for test automation.

What Will I Learn ?

1. How to ensure instructions in a test are executed when the application is ready.
2. How to implement different synchronization functions from the Squish API and know when they might be useful.
3. How to communicate with Qt's signals and event handler system to synchronize testing.

[How to Create and Use Synchronization Points | Squish 9.2]
[How to Use Event Handlers | Squish 9.2]
[Qt Convenience API | Squish 9.2]
[GitHub - qt-learning/Test-Synchronization · GitHub]

When running a test script using Squish, it is important that the **test script and Application Under Test (AUT) remain synchronized**. This will ensure that the test script does not send commands to the AUT that the AUT is not ready to carry out.

For example, the AUT contains a dashboard that can only be accessed once a user has logged into their profile. If the test script is written to log in to a profile and to try to immediately interact with the dashboard, it is possible that the script and the AUT get out of sync. This is because the AUT may take time to validate the profile credentials and not make the dashboard available immediately. This could result in Squish throwing an unanticipated error, as the script would have attempted to interact with a dashboard that is not yet available. In fact, almost any operation that causes even a slight delay in the AUT running, such network latencies or re-loading of components, could cause such synchronization issues to occur. For this reason, the Squish API provides several methods that ensure that the test script and the AUT remain synchronized.

We will go over these methods, including:

- Time-Based Synchronization
- Object-Based Synchronization
- Image-Based Synchronization
- Text-Based Synchronization
- Conditional Synchronization
- Event and Signal Handlers

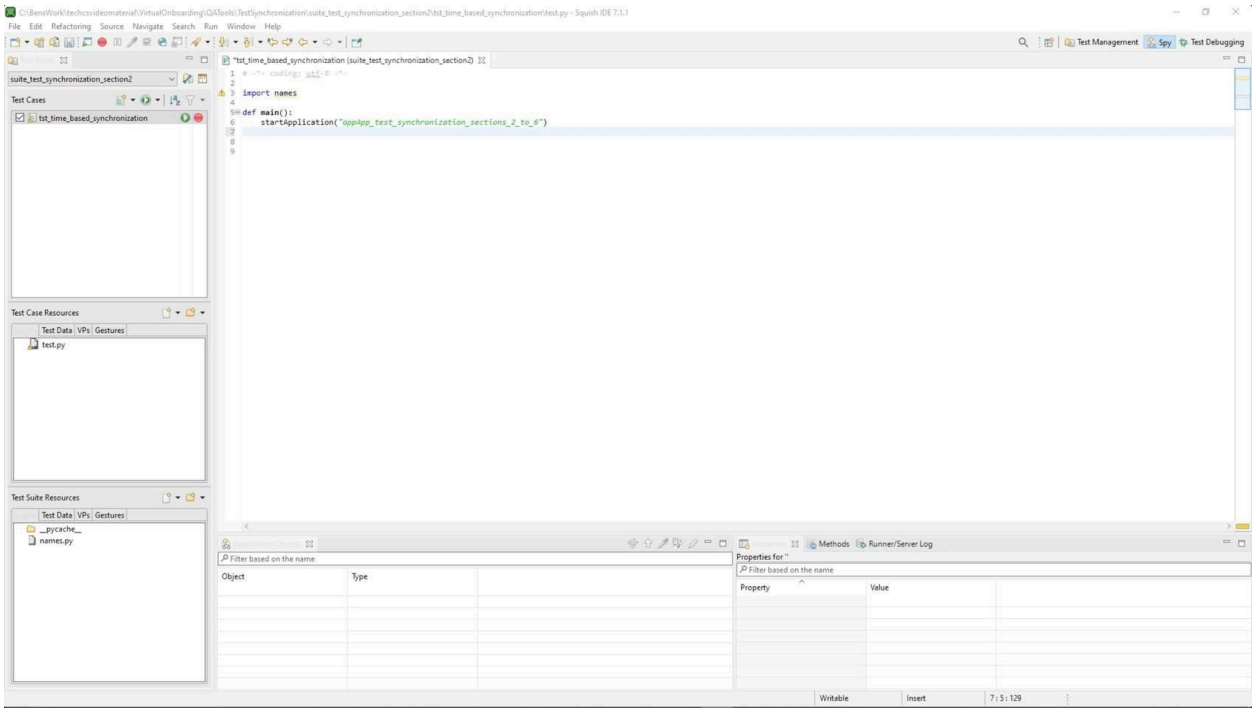
Time-Based Synchronization

The first and most straightforward method of test synchronization is **time-based synchronization**, also known as fixed-time synchronization. Time-based synchronization is a form of test synchronization where the snooze(seconds) function is inserted into the test in places where the AUT may need time to run some operation, like when waiting for a window to pop up.

```python
mouseClick(names.test_Sync_Example_Open_Button)
snooze(1)
mouseClick(names.test_Sync_Example_Button)
```

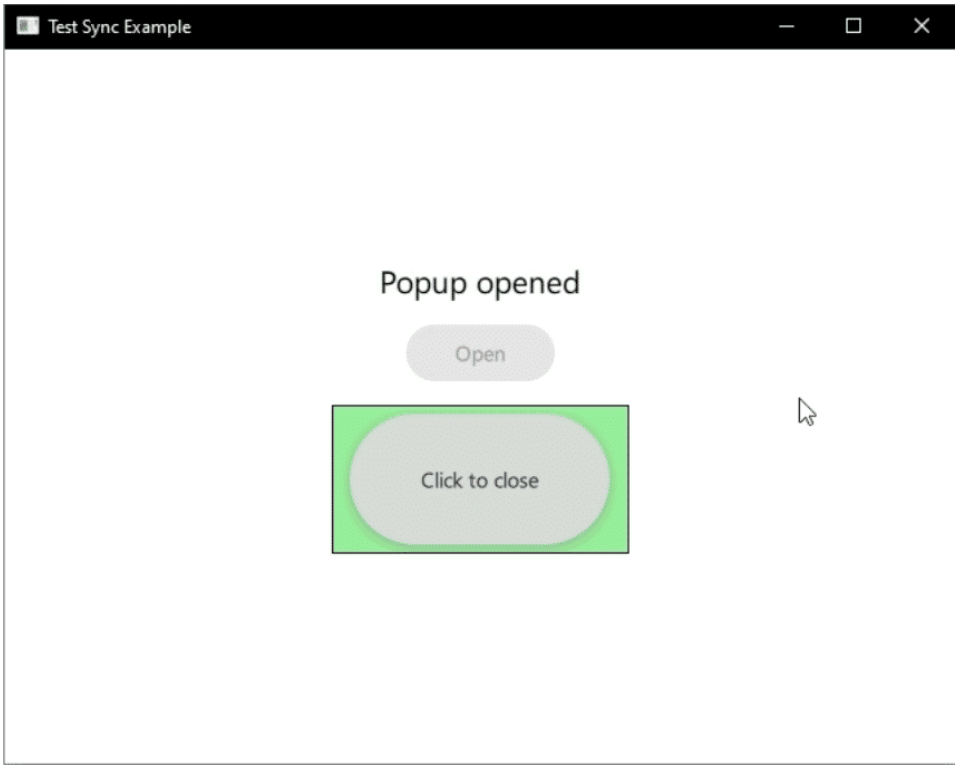
STEPS

01 Check the test app is built and mapped.



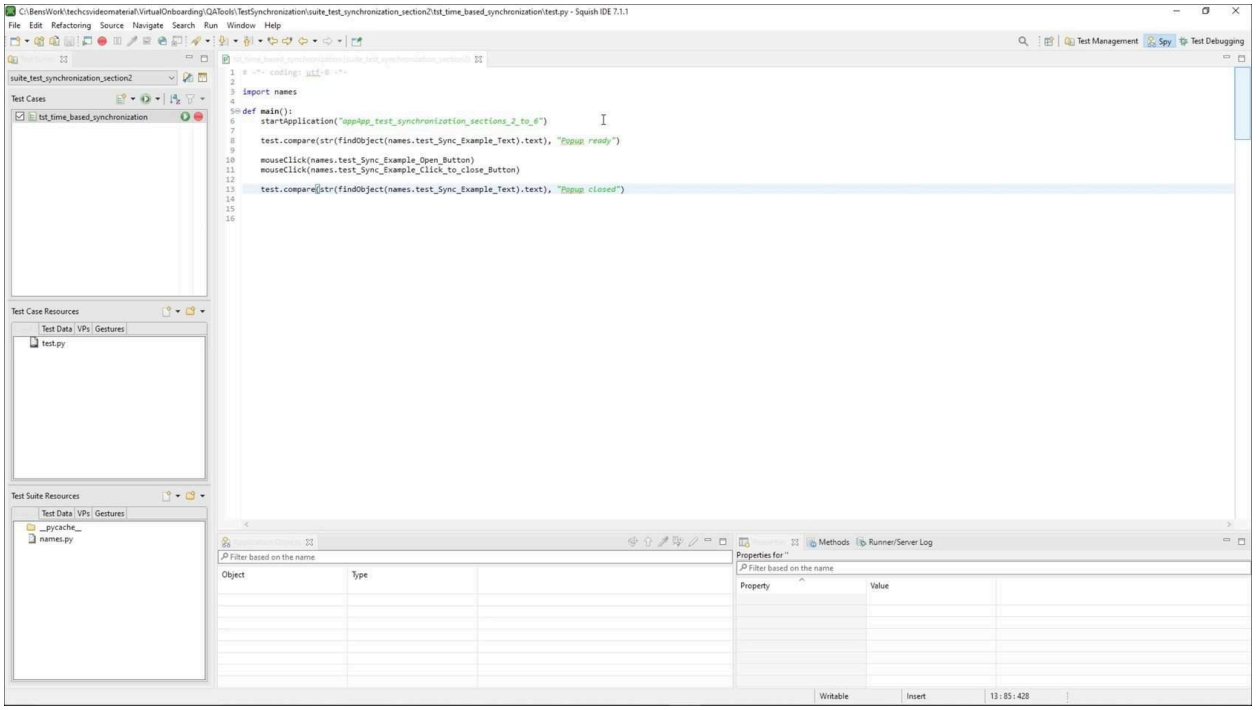
For this demo, we used the app located in 'App_test_synchronization_sections_2_to_6' folder of the course repo. This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

02 Understanding the app



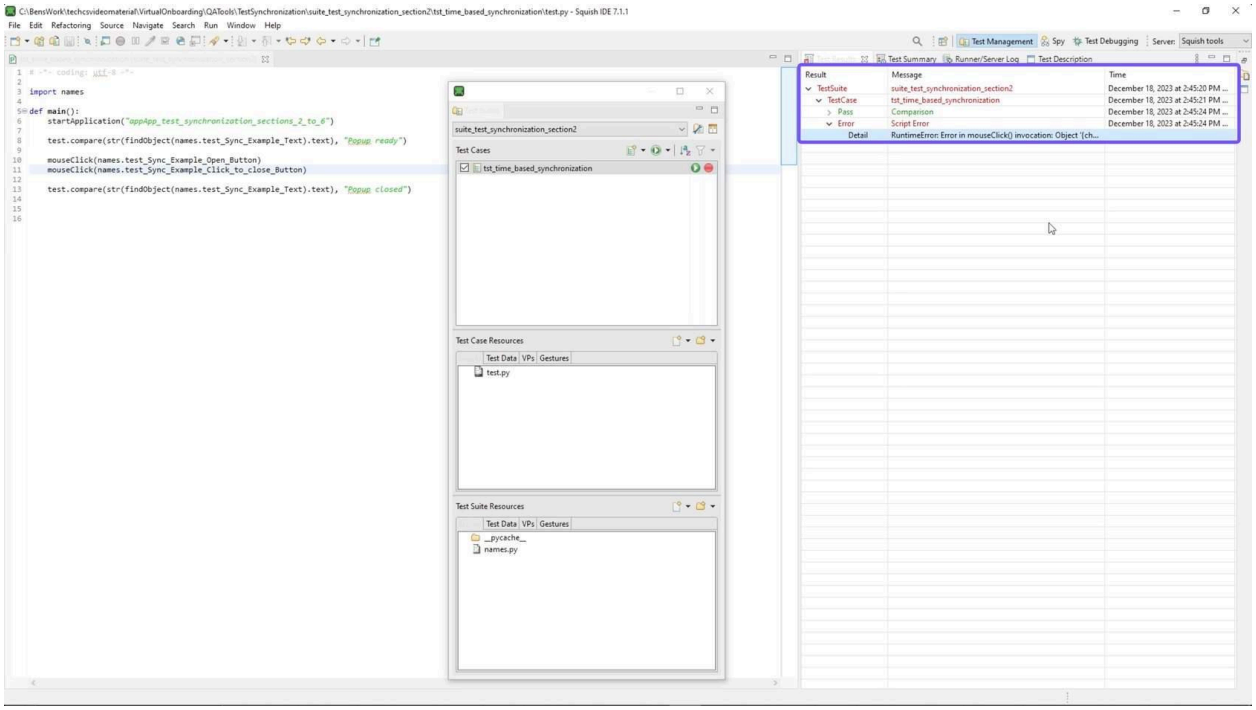
The application showcases a scenario where a component loads at runtime, taking a significant amount of time. Upon running the application, clicking the "Open" button changes the text to "Popup opening...". After a brief loading period, the text updates to "Popup opened", accompanied by a new button. Clicking this new button closes the popup, changing the text to "Popup closed".

03 Implementing the test



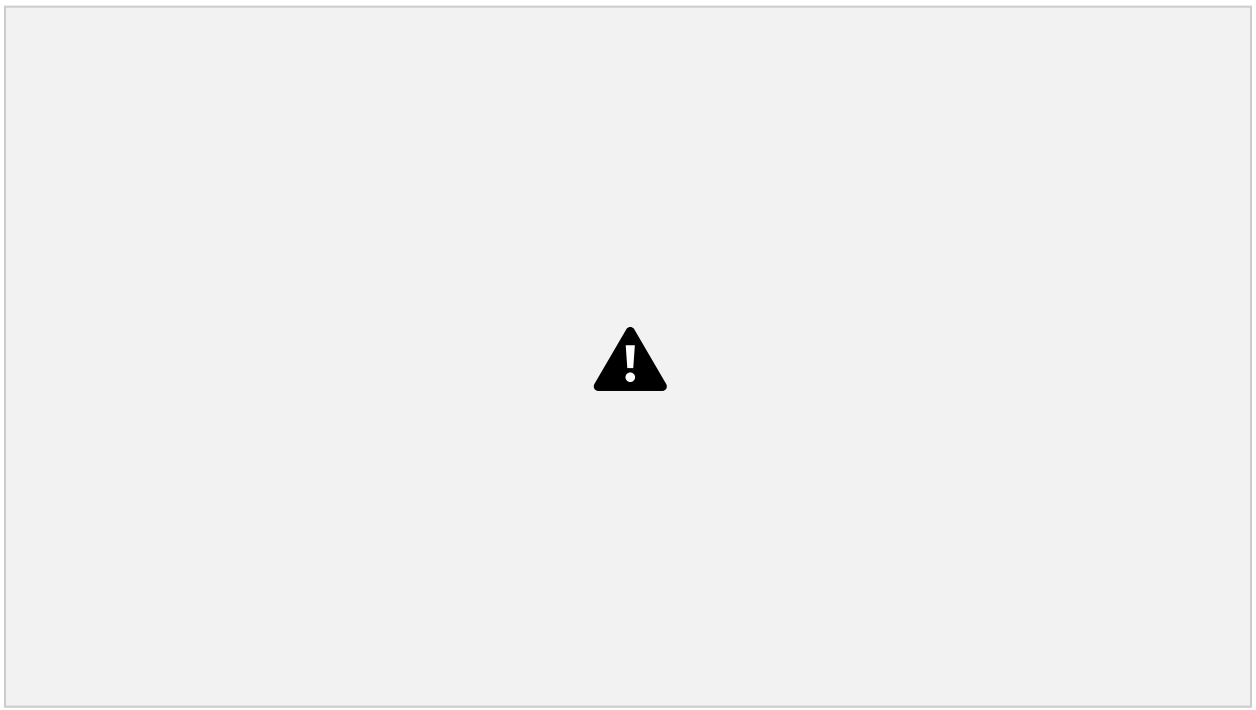
We added code to compare the initial status text with the expected "Popup ready" and then used the mouseClick() function to open the popup, then the same again to close the popup with the close button. We can now compare the status text string to our predefined value "Popup closed" to see if the status text has been updated correctly.

04 Addressing the test error



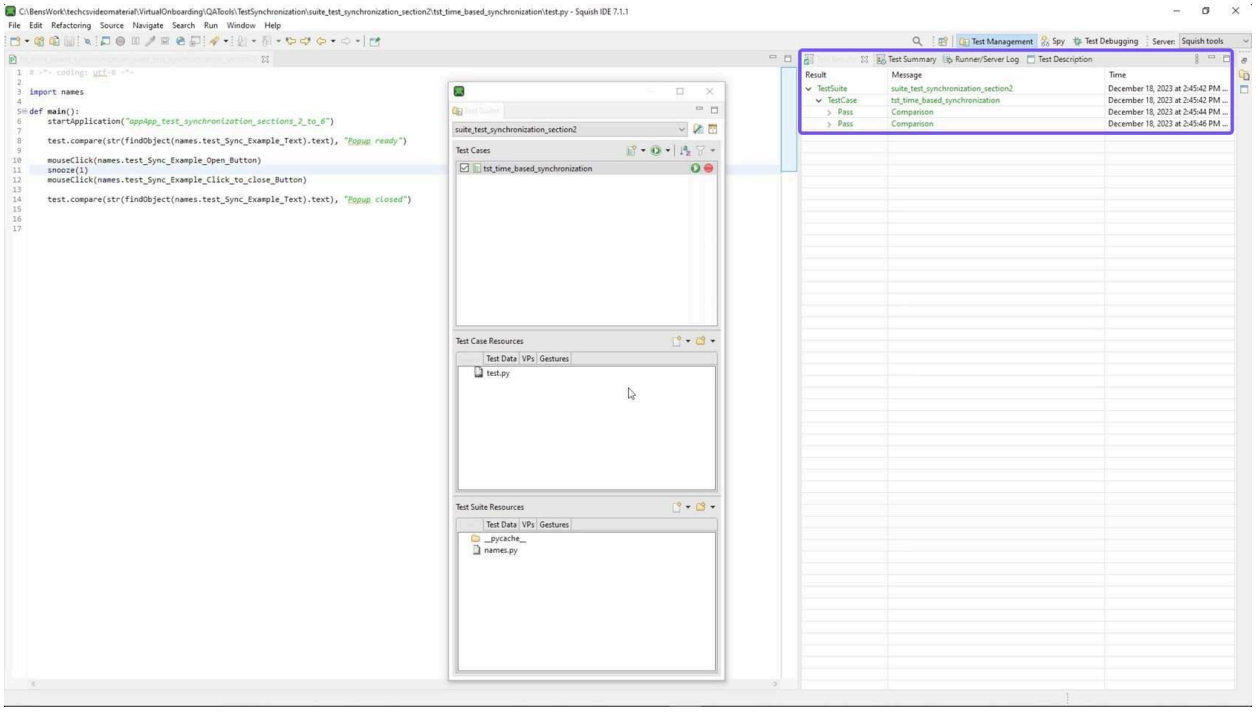
Our initial test run failed due to premature interaction with the popup, which hasn't loaded fully. Our test tried to interact with the popup before it was available, resulting in an error. This issue is identified as an "Error in mouseClick() invocation".

05 Using fixed-time synchronization



To resolve this, we can use fixed-time synchronization. By adding a one-second snooze between the mouse clicks, the test script waits before interacting with the popup, ensuring it's fully loaded.

06 Re-running the test



After implementing the fixed-time synchronization, we can rerun the test. This time, it successfully passes.

Object-based Synchronization

The first form of variable-time synchronization is **object-based synchronization**. Object-based synchronization is a method where Squish will wait for an object to “exist” before attempting to interact with it. This is almost always preferable to time-based synchronization, as Squish will wait the exact amount of necessary time before attempting to interact with an object. Because of this, object-based synchronization is the default method Squish uses when recording a test case.
The Squish API provides three functions for object-based synchronization:

1. **waitForObject(objectOrName, [timeoutMSec])**
    - Waits for the given object to exist, be accessible (i.e., visible) and be enabled, then returns a reference to that object.
    - Raises an exception failure if time out is reached.
    - The timeout parameter is optional, when not defined it takes the timeout defined in the Test Suite settings (20 seconds by default).

```
mouseClick(waitForObject(names.test_Sync_Example_Open_Button, 1000))
```

2. **waitForObjectExists(objectOrName, [timeoutMSec])**
    - Waits for the given object to exist and be enabled, then returns a reference to that object.
    - Does not require for the object to be visible, so it is best used with non-visible objects.
    - Raises an exception failure if time out is reached.
    - The timeout parameter is optional, when not defined it takes the timeout defined in the Test Suite settings (20 seconds by default).

```
test.compare(str(waitForObjectExists(names.test_Sync_Example_Text).text), "Popup closed")
```

3. **waitForObjectItem(objectOrName, itemOrIndex, [timeoutMSec])**
    - Waits for the given object to exist, be accessible (i.e., visible) and be enabled, and contains an item that is identified by the itemOrIndex parameter.
    - Once found it will return a reference to that object.
    - Raises an exception failure if time out is reached.

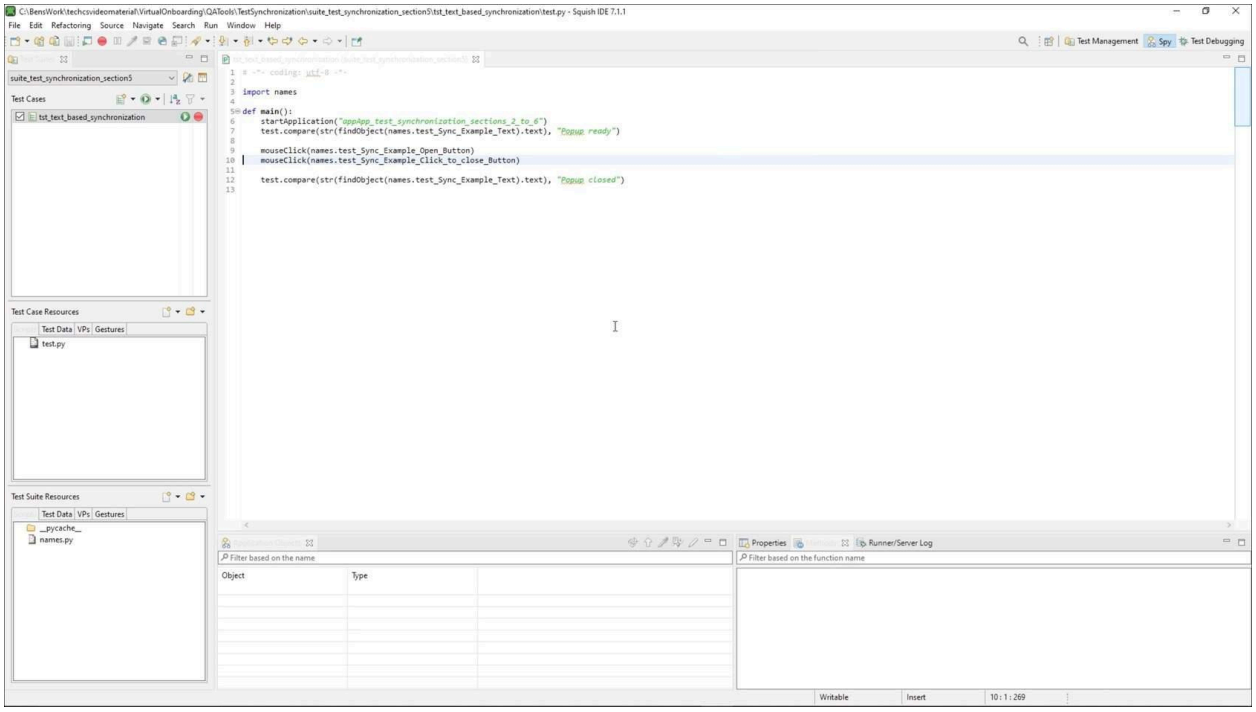
- This function should be used primarily to identify objects inside container objects such as tables or lists.
- The timeout parameter is optional, when not defined it takes the timeout defined in the Test Suite settings (20 seconds by default).

```
mouseClick(waitForObjectItem(names.address_Book_MyAddresses_adr_File_QTableWidget, "0/0"))
```

If possible, one of these functions should be used any time an object reference is made in a test script.

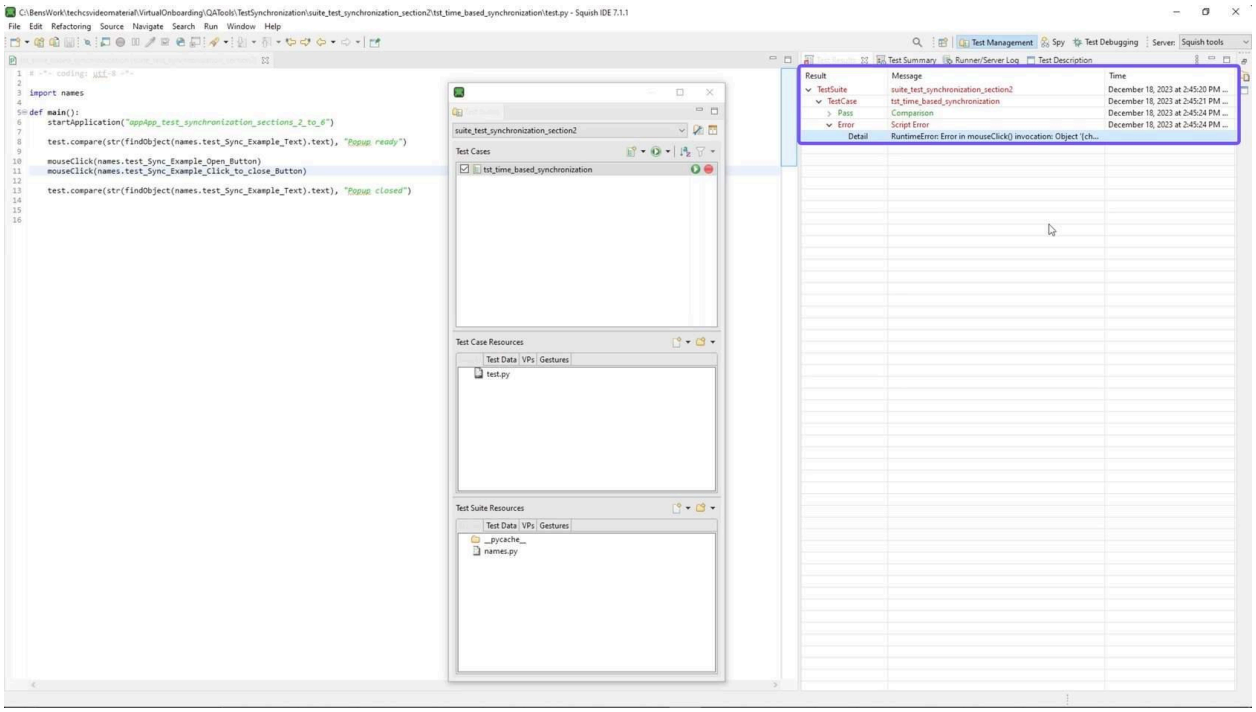
STEPS - Object-Based Synchronization in Action

01 Test application



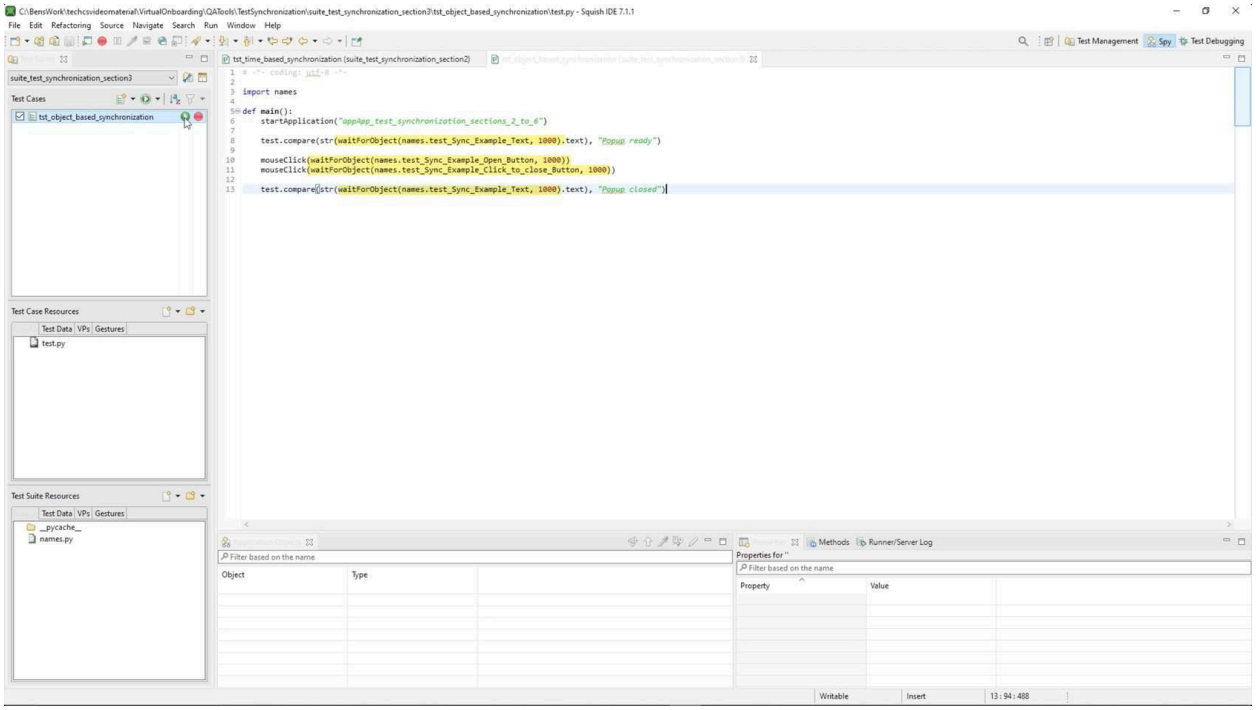
For this demo, we used the app located in 'App_test_synchronization_sections_2_to_6' folder of the course repo. This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

02 Switching to object-based synchonization.



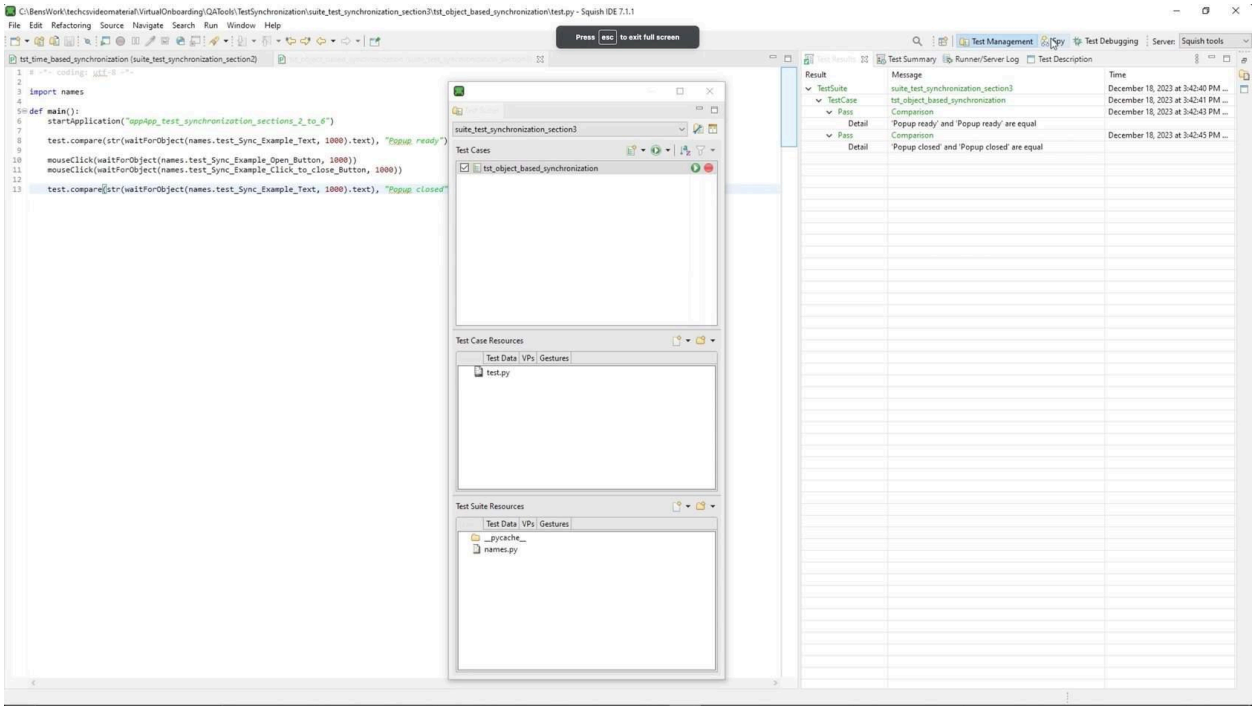
Starting with the app and test code from the last section, we checked the status text, clicked to open a popup, waited, closed the popup, and verified the status text update. We will start by removing snooze() and running the test to remind us of what happens. The test fails as the close button does not yet exist.

03 Adding waitForObject()



The waitForObject() function will pause the test for the specified timeout duration, ensuring the test pauses until the referenced object is available. The key advantage here is that the test waits only as long as necessary for an object to appear, speeding up the test compared to fixed-time synchronization.

04 Running the test



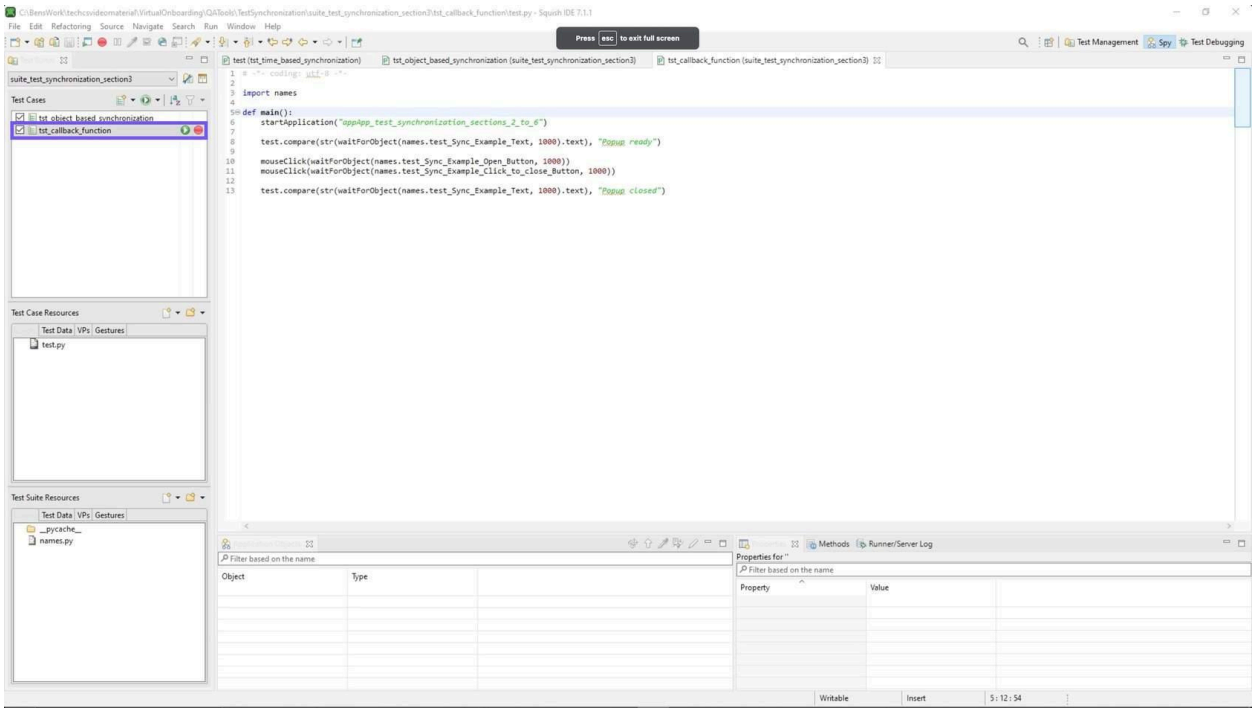
Running the test with these modifications, we observe a successful pass. The script waits only as long as necessary before interacting with the popup button. When using the Squish IDE's recording tool, functions like waitForObject() and its counterparts are automatically inserted into object references, making it a quick way to start setting up your tests.

Callback Functions

For the most part, these functions will be sufficient for most object-based synchronization needs. However, it can at times be useful to implement additional functionality alongside these functions. For example, it may be useful to highlight an object, or do some additional processing any time it is detected with the waitForObject(objectOrName) function. This can be done by implementing a callback function called waitUntilObjectReady(anObject). This callback function will be executed immediately after the waitForObject(objectOrName) anytime it is called, allowing for custom functionality to be implemented using the object being detected. The waitForObjectItem(objectOrName, itemOrIndex) function can also be customized in the same way using a callback function called waitForObjectItemReady(Item).

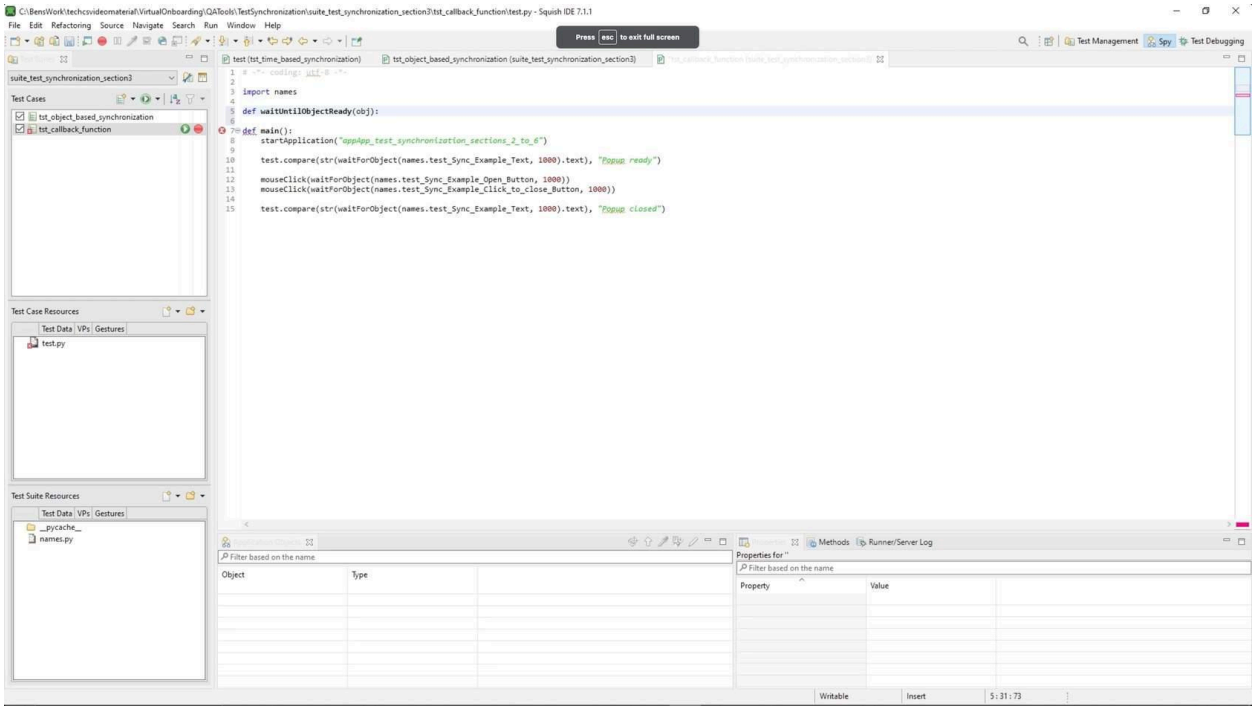
STEPS - Exploring Callback Functions

01 Setting up the test script



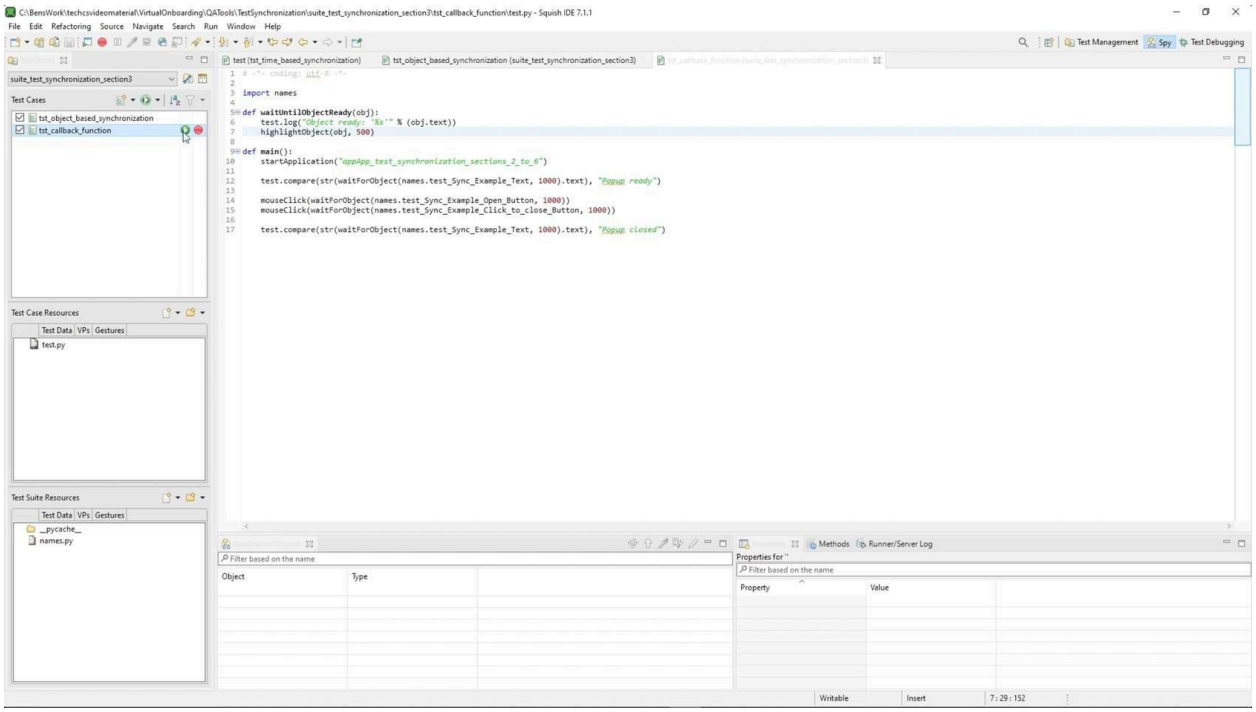
We begin with duplicating our previous test script, now named “tst_callback_function”. This script retains the same test code. We will use this as a basis to implement the callback function.

03 Defining the callback function



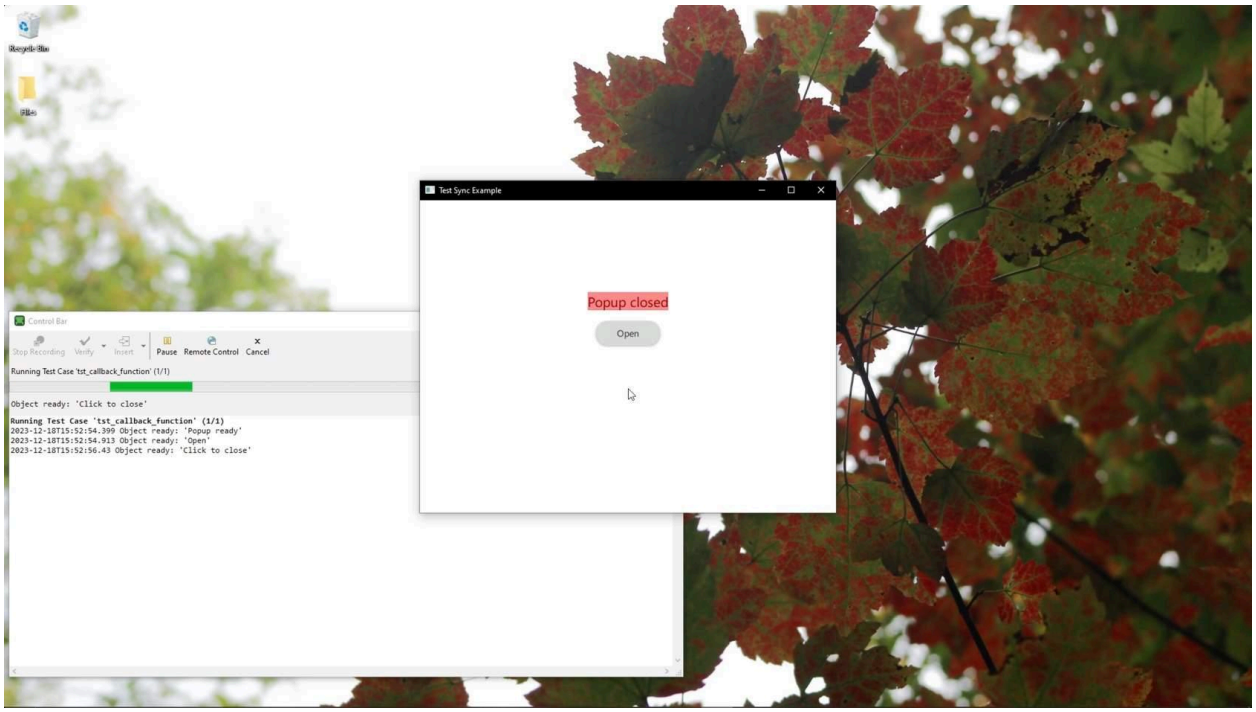
Start by defining the function, waitUntilObjectReady() in the test script, passing the object as a parameter. This function is designed as a callback to be executed immediately following each waitForObject call, allowing us to customize the behavior of the waitForObject function. This is a reserved function in the Squish API and will be executed immediately after the waitForObject function is called, every time it is called.

03 Implementing the callback function



We can now implement the function and customize the behavior of waitForObject function by implementing addition functionality in our callback function. In our case, we will print out a message indicating that the object passed is ready and visually highlight the object for 500 milliseconds.

04 Running the test



Each object is highlighted in red just after the waitForObject function is called. In the test results view, we observe logs detailing the text of each object as it is found by the waitForObject function.

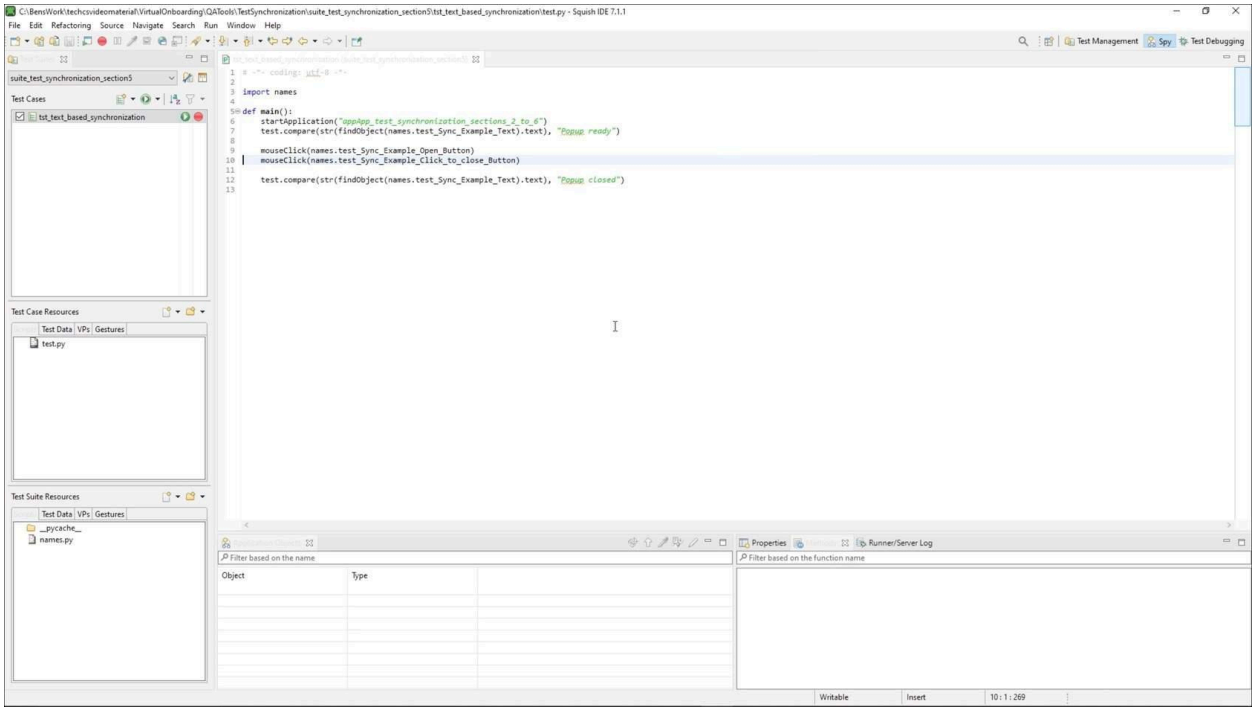
Image-Based Synchronization

Image-based synchronization is conceptually similar to object-based synchronization. However, instead of waiting for an object to be available, it waits for an image to be visible. Image-based synchronization uses the waitForImage(imageFile, [parameterMap], [searchRegion]) function. This function will wait for an image matching the imageFile to appear on the screen until a specified timeout has been reached. Once this image has been located, it will return a ScreenRectangle, which will identify the location of the found image. The optional parameterMap and searchRegion parameters can be used to customize the behavior of the image-based synchronization further.

```python
mouseClick(waitForImage("OpenButton",{ "tolerant": True, "threshold": 99.999 },
  waitForObjectExists(names.test_Sync_Example_QQuickWindowQmlImpl)))
```

STEPS

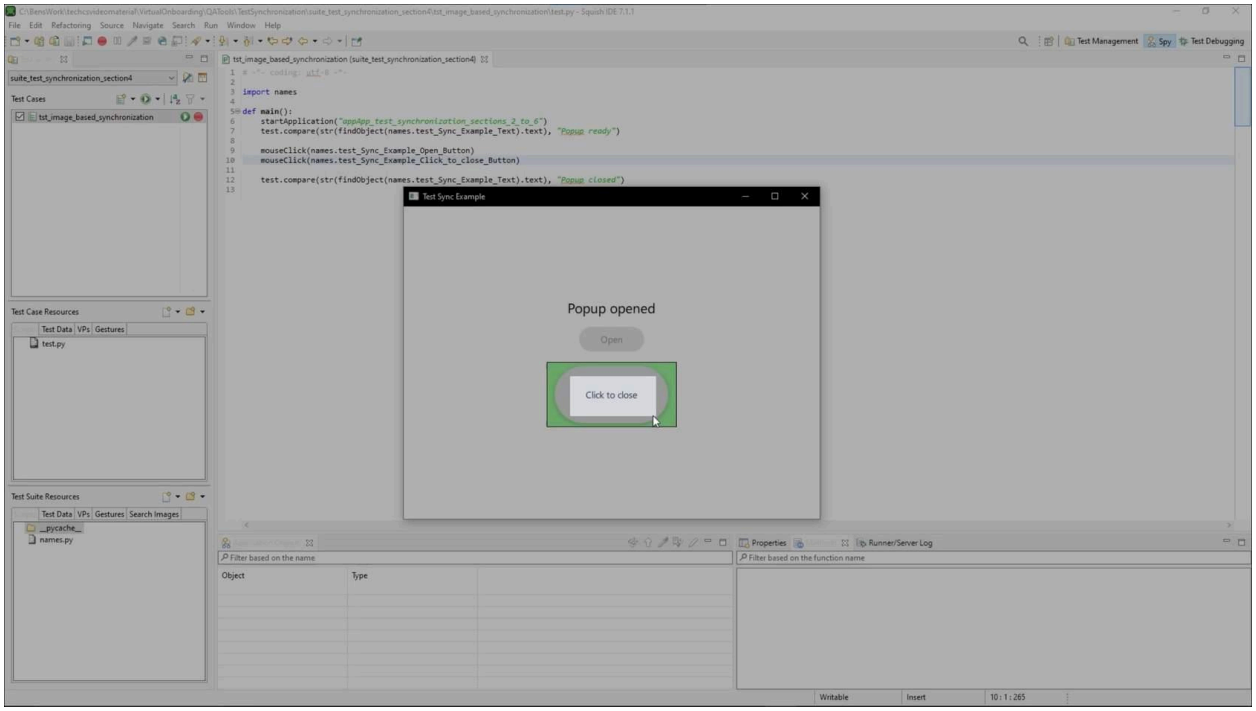
01 Test application



For this demo, we used the app located in 'App_test_synchronization_sections_2_to_6' folder of the course repo. This application was created for this demonstration and must be built for the

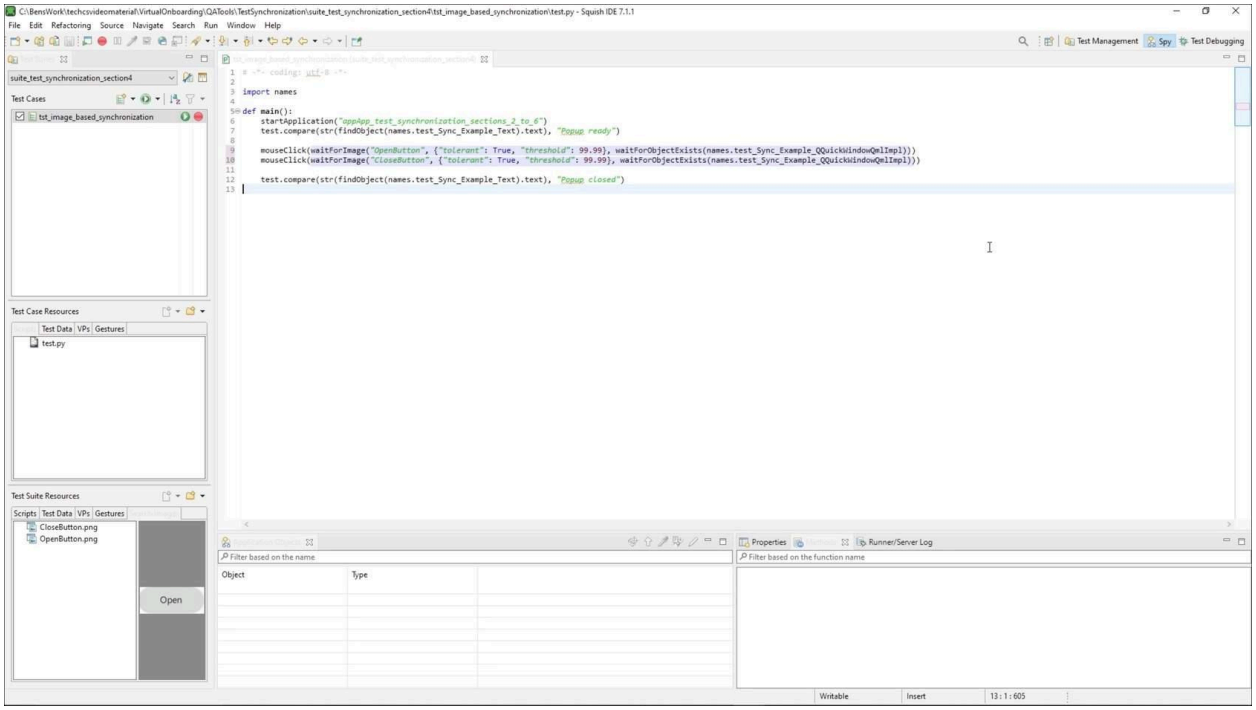
specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

02 Capture screenshots



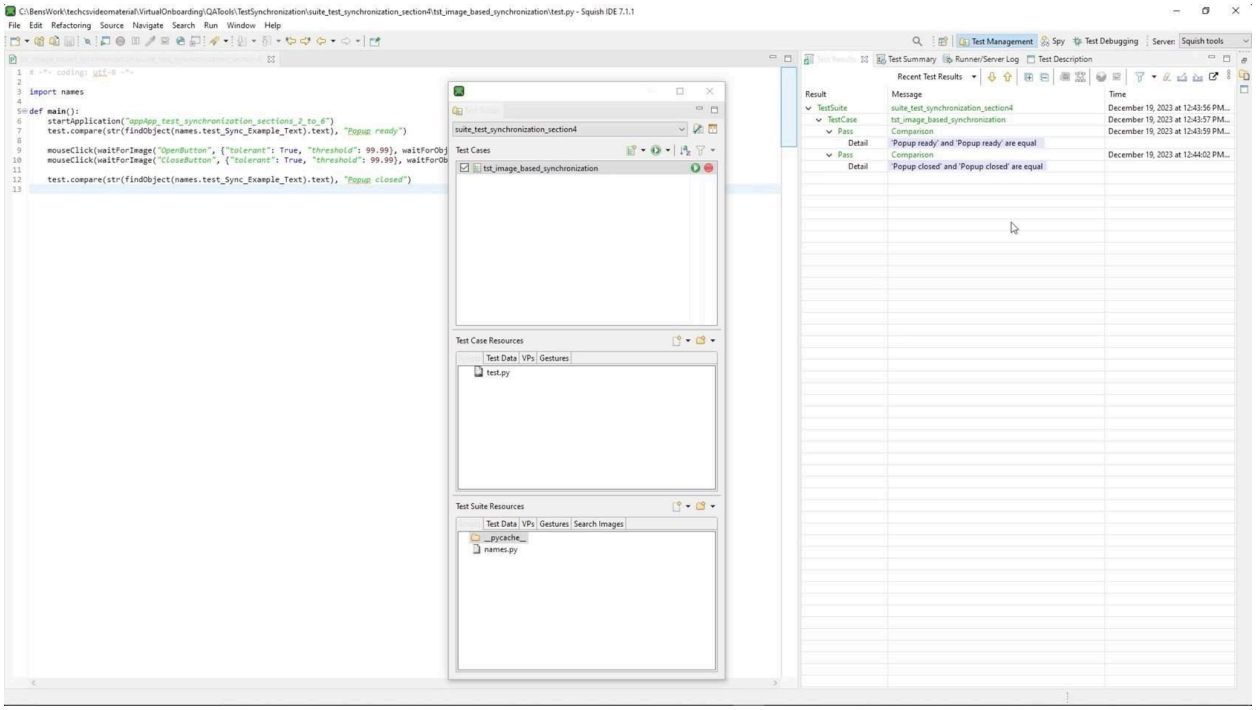
We need some screenshots of the application objects we intend to interact with. So, we will run the executable for the application outside of the Squish IDE. We will capture a screenshot of the open and close button and name them "OpenButton" and "CloseButton" respectively. Move the images to the shared/searchimages folder.

03 Implementing image-based synchronization



Starting with the previously created test with no synchronization, we can modify our test code to include image-based synchronization points using the waitForImage() function. The function will continuously scan our display for an occurrence of an image that matches the referenced image until it is found or the timeout is reached. It will return the location of the image when found, which can be given directly to the mouseClick() function to click on. We will use the overload, which takes three parameters - The image we would like to search for, toggling tolerant search with a threshold, and a region for us to search, we will reference the entire window. We will implement this for the open and close button.

04 Running the test



Running the test with the waitForImage() function, we see that the test successfully completes with the image-based synchronization implemented.

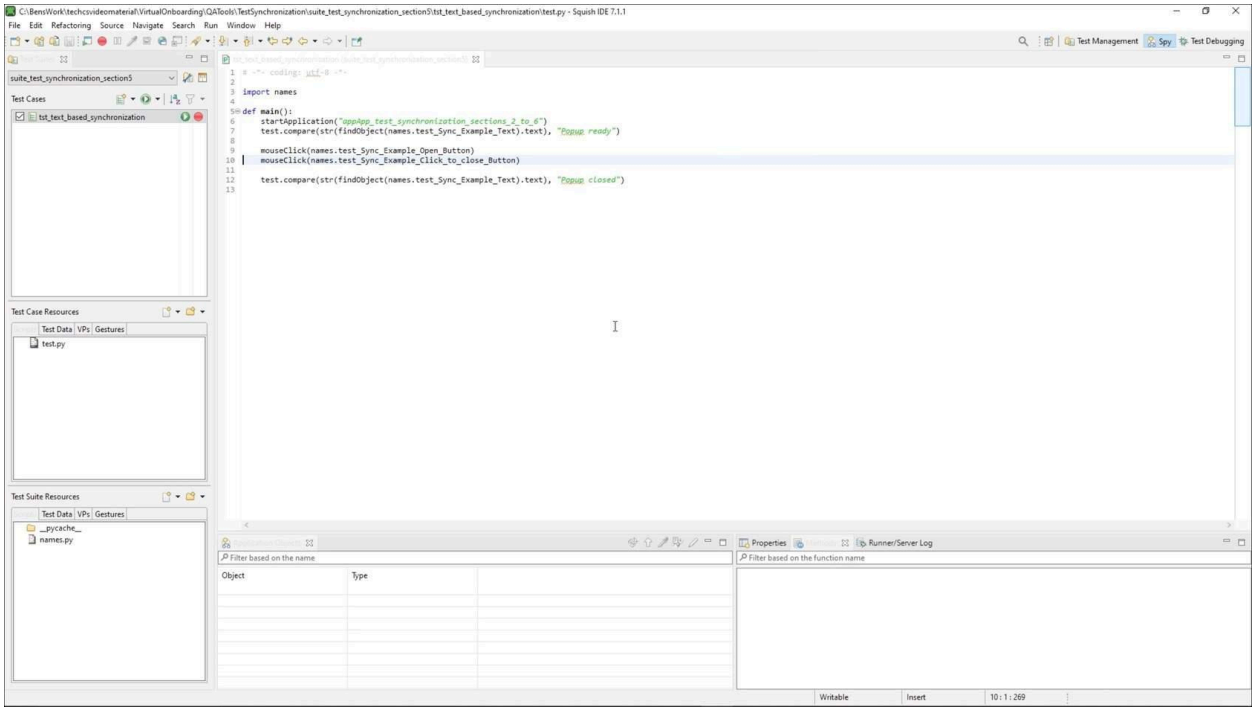
Text-Based Synchronization

Text-based synchronization is also similar to object-based synchronization and image-based synchronization. It has the key difference, however, that it relies on an external engine for text recognition, Tesseract being the recommended engine. With Tesseract (or another equivalent engine) installed, the waitForOcrText(text, [parameterMap], [searchRegion]) function can be used. This function will perform repeated Optical Character Recognitions (OCR) on the display until the specified text is found, returning a ScreenRectangle when found, or raising an exception failure if the timeout is reached. The optional parameterMap and searchRegion parameters can be used to customize behavior of the text-based synchronization further.

```python
mouseClick(waitForOcrText("Open"))
```

STEPS - Exploring the waitForOcrText() Function

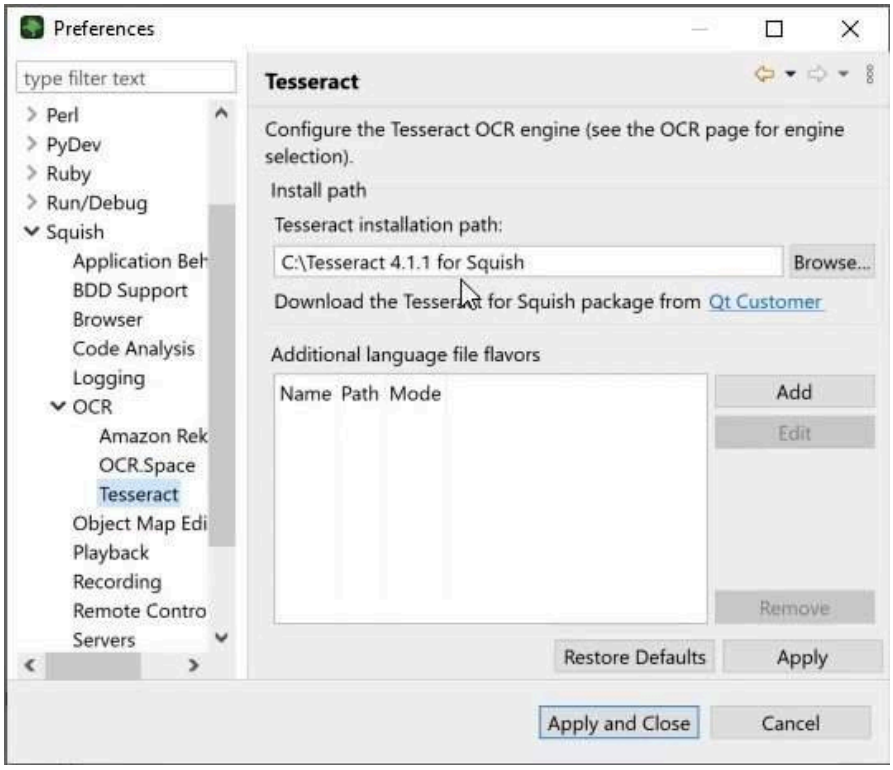
01 Test application



For this demo, we used the app located in 'App_test_synchronization_sections_2_to_6' folder of the course repo. This application was created for this demonstration and must be built for the

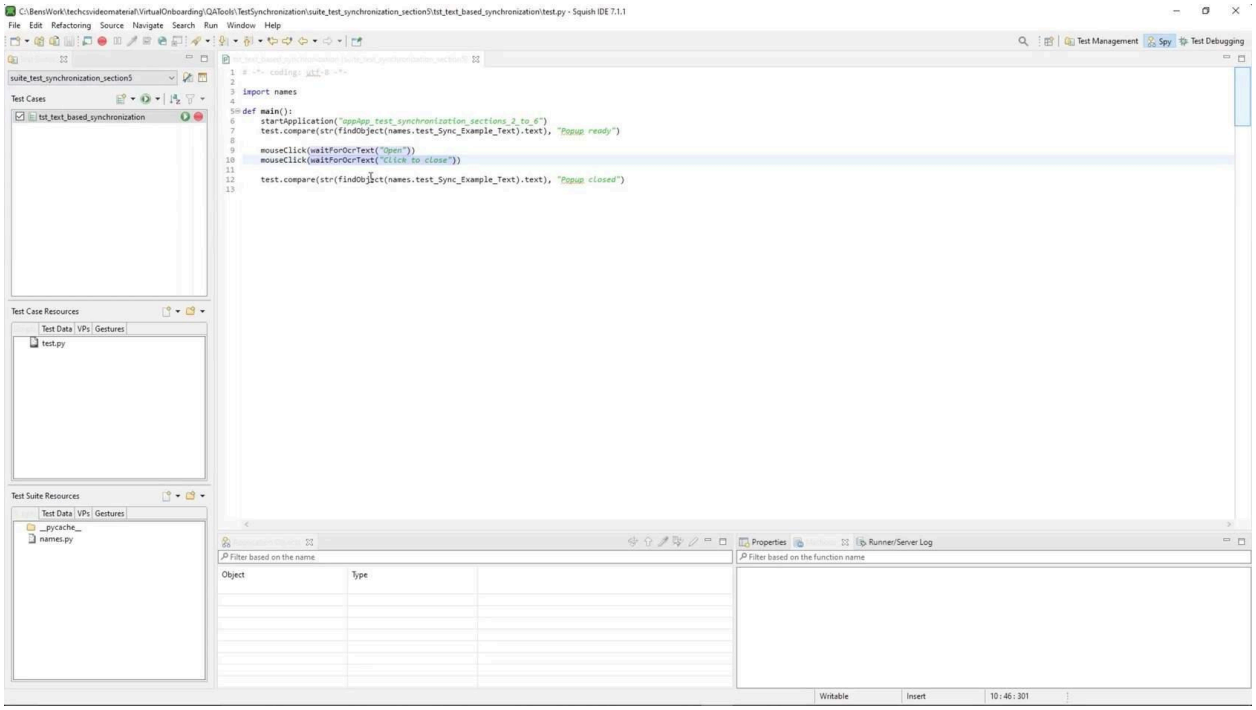
specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

02 Using OCR in Squish



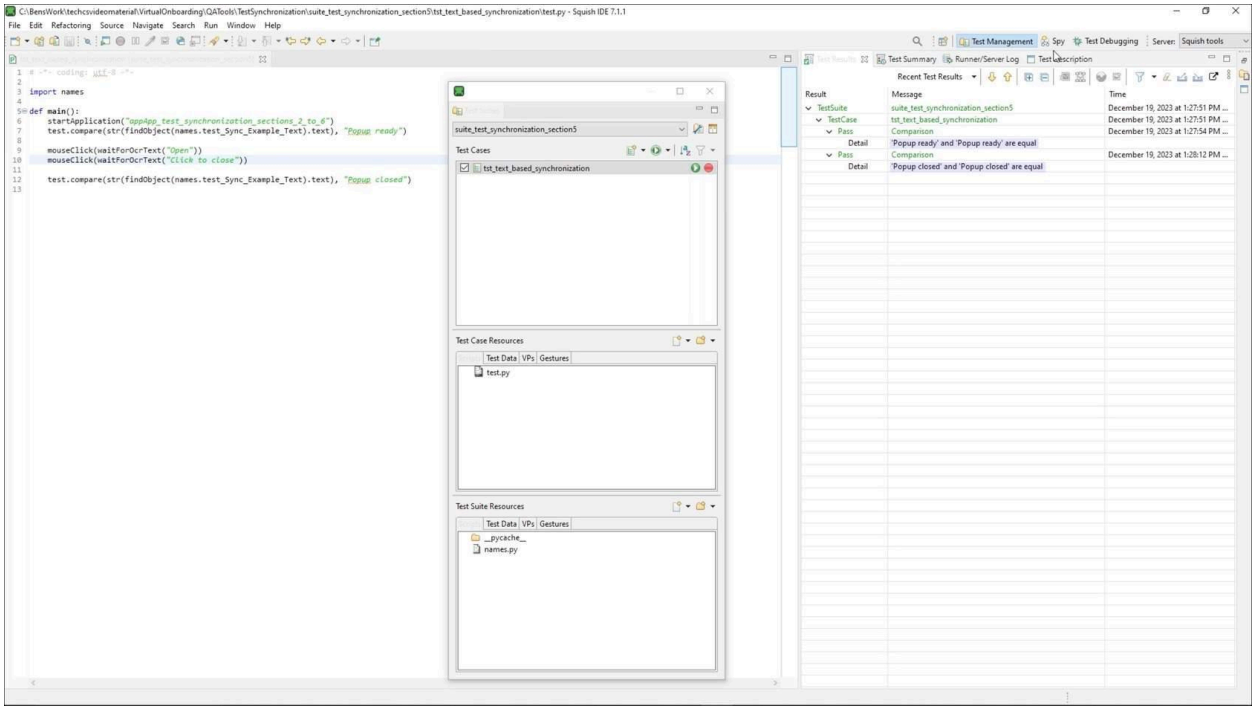
Starting with the test written previously, which does not use any synchronization, we will use the **waitForOcrText()** function. For this function to work, a third-party engine must be installed. **Tesseract for Squish** is the primary engine, which can be downloaded from the Qt Customer Portal.

03 Implementing waitForOcrText



The **waitForOcrText()** function will continuously scan our entire display for an occurrence of text which matches the referenced text until it is found, or the timeout is reached. It will return the location of the text when found, which can be given directly to the mouseClick function to click on. In our case, **waitForOcrText("Open")** to wait for and interact with the open button and **waitForOcrText("Click to close")** once the popup has opened and the button is visible.

04 Running the test



Our test now passes without any issues Since our open button has the text “Open” and close button has the text “Click to close”, Squish will click these buttons once they become visible.

Conditional Synchronization

Sometimes setting a synchronization point based on the presence of objects, images, or text is not satisfactory, but instead a synchronization point based on a condition is much more useful. For example, let us say an AUT has a component which can exist in an “on” or “off” state which does not show any visible difference in the UI. If the test should only continue once that component is in the “off” state, then none of the previously discussed synchronization methods would work, as they all check for the presence of something but do not take its state into account. In this situation, conditional synchronization is optimal. This is because the synchronization point should wait for some condition to be fulfilled, such as the component being in the “off” state, before continuing.

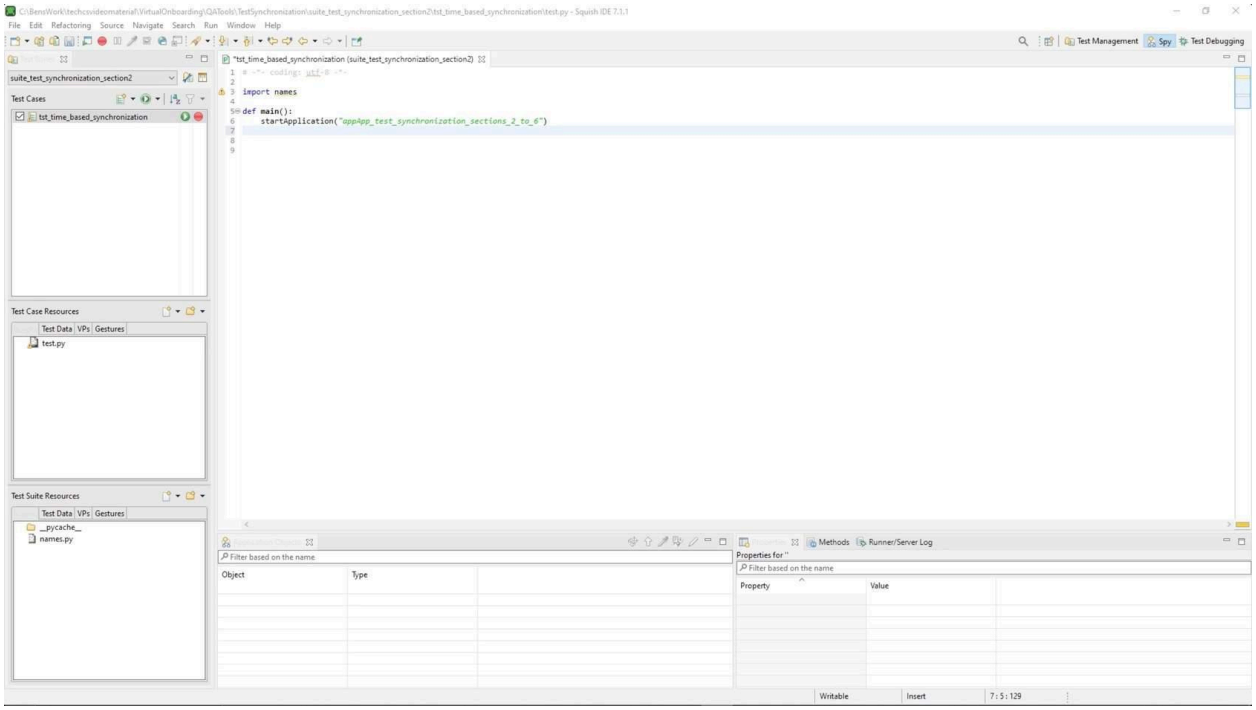
Conditional synchronization is done using the waitFor(condition) function. This function will repeatedly evaluate the condition until either the condition is evaluated to be true, or an optional timeout is reached, returning true or false, respectively. The condition is simply a piece of code written out as a string, and can therefore be any condition, as long as it returns a Boolean. It is important to note that the **waitFor(condition)** function has no default timeout. The example below demonstrates how this can be done by checking the value of a status string, which confirms the presence of a pop-up before clicking on it.

None

```
obj = waitForObjectExists(names.test_Sync_Example_Text)
if waitFor("str(obj.text) == 'Popup opened'", 5000):
        mouseClick(names.test_Sync_Example_Click_to_close_Button)
    else:
        test.log("waitFor() timeout reached")
```

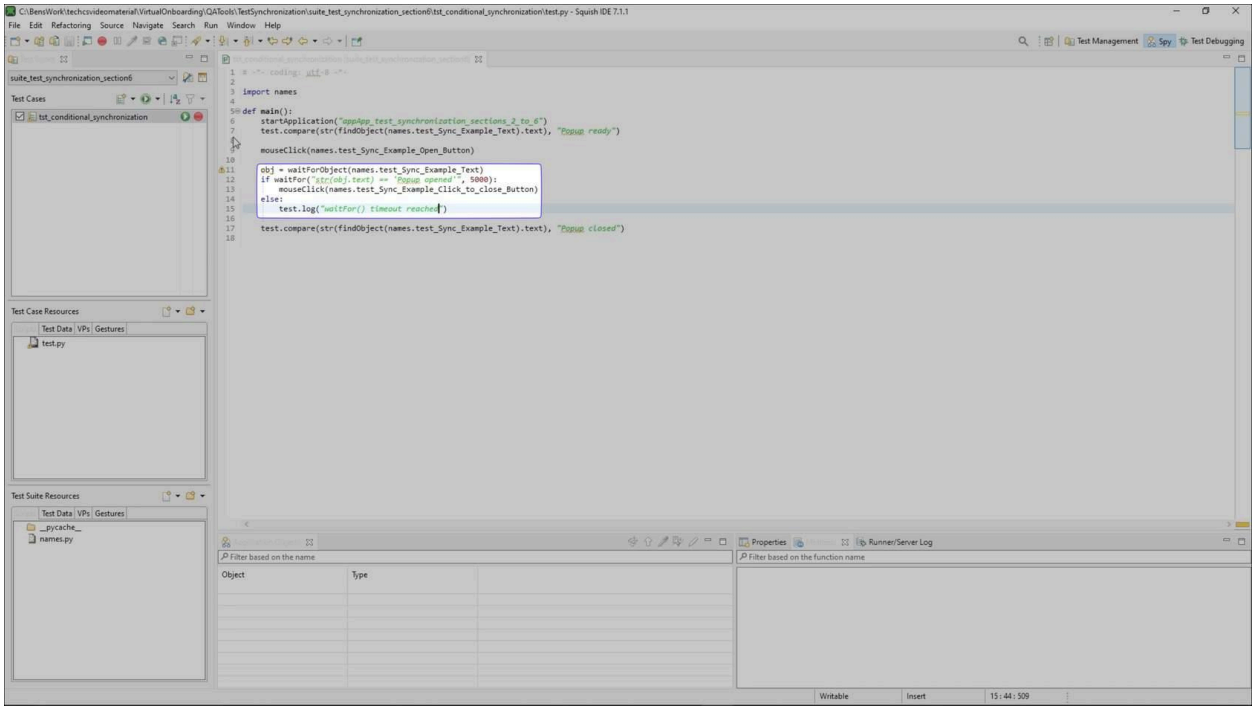
STEPS

01 Test application



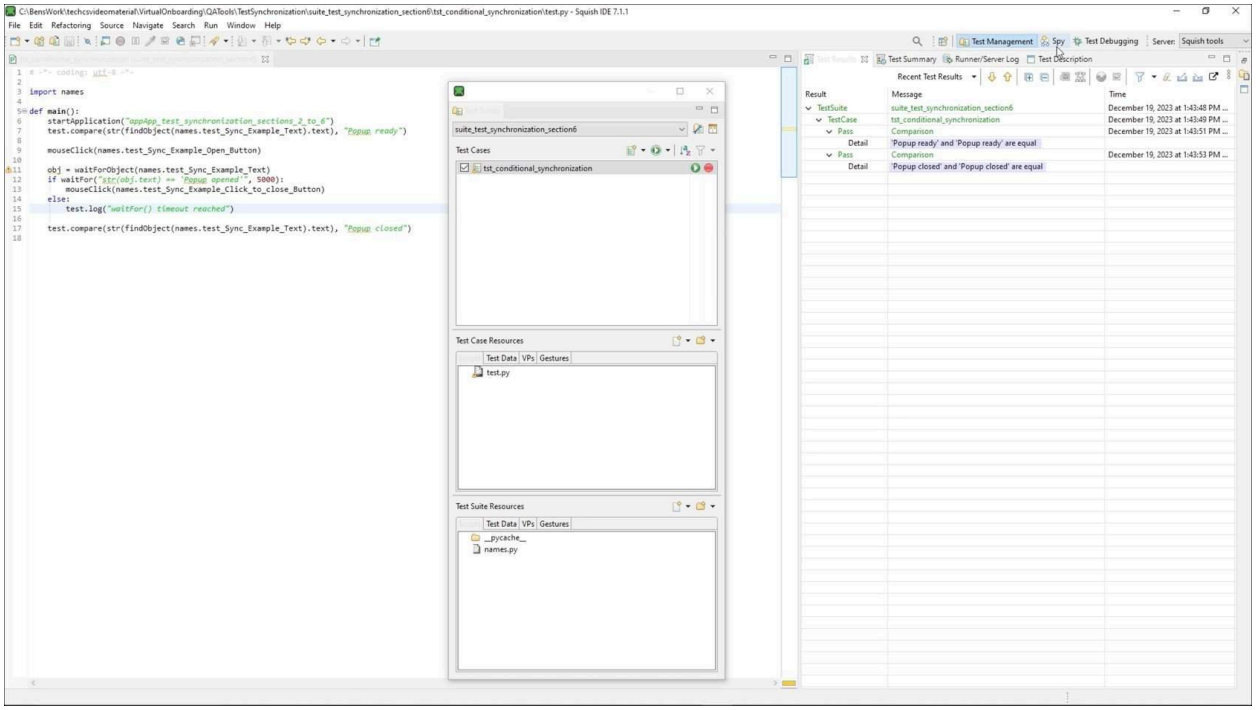
For this demo, we used the app located in 'App_test_synchronization_sections_2_to_6' folder of the course repo. This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit. We will start with the test written previously, which does not use any synchronization.

02 waitFor function

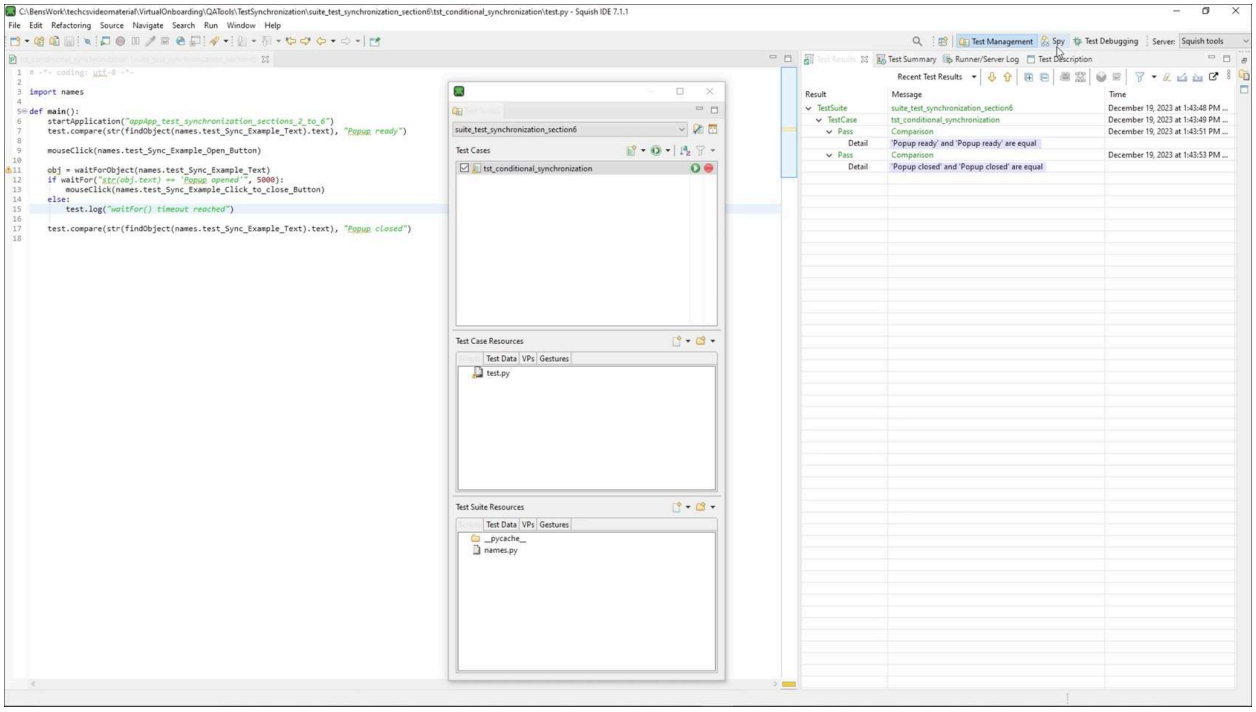


The **waitFor()** function continuously evaluates a given condition until it returns true or a specified timeout is reached. We will use the change in status text to "popup opened" as the trigger condition for further actions. We first store the status text and can reference this in our conditional statement. If the timeout is reached without the condition being satisfied, a log message is generated that the **waitFor()** has timed out.

03 Running the test



Our test now passes without any issues. It is important to note that the condition inside the waitFor() function can be a Boolean property, complex script statement, or function reference. As long as the condition returns a Boolean value, it is valid to use inside the waitFor() function.



```python
# -*- coding: utf-8 -*-

import names

def main():
    startApplication("appApp_test_synchronization_sections_2_to_6")
    test.compare(str(findObject(names.test_Sync_Example_Text).text), "Popup ready")

    mouseClick(names.test_Sync_Example_Open_Button)

    obj = waitForObject(names.test_Sync_Example_Text)
    if waitFor("str(obj.text) == 'Popup opened'", 5000):
        mouseClick(names.test_Sync_Example_Click_to_close_Button)
    else:
        test.log("waitFor() timeout reached")

    test.compare(str(findObject(names.test_Sync_Example_Text).text), "Popup closed")

```

Event and Signal Handlers

The final method of test synchronization works slightly differently from the others. This method relies on the usage of event handlers or signal handlers. These handlers allow the test to react directly to the application's signals and event handlers. For example, an event handler can execute code when a specific AUT event occurs, such as a dialog being opened or the AUT crashing. Signal handlers, on the other hand, allow the test to execute code when a specific signal is emitted, such as when a button is clicked, or a slider value is changed.
Both handler types are defined and installed in a comparable way, but have some key differences.

Event Handlers

For event handlers, a handler function with the correct parameters must first be defined outside the main test function of the test. This function will be executed anytime the event occurs, and can do whatever the test requires, such as printing out logging information. The parameters of this function depend on whether it is a global event handler or an object specific event handler. Global event handlers will interact with events that affect the entire application. These global events will sometimes pass parameters to the event handler that they are associated with. When that is the case, a parameter for event handler function should be included.

Once the event handler function has been created the event handler must be installed. This is done using the installEventHandler(eventName, handlerFunctionName). This function should be executed in the main test function, and once executed will cause the handlerFunctionName to be executed any time the specified eventName occurs. The eventName can be any of the Squish convenience event types for a Qt application:

1. Crash - occurs if the AUT crashes.
2. DialogOpened - occurs when a top-level QDialog is shown
3. MainWindowOpened - occurs when a top-level QMainWindow is shown
4. MessageBoxOpened - occurs when a top-level QMessageBox is shown
5. Timeout - occurs when the Squish response timeout is reached
6. ToplevelWidgetOpened - occurs when any other kind of top-level widget is shown

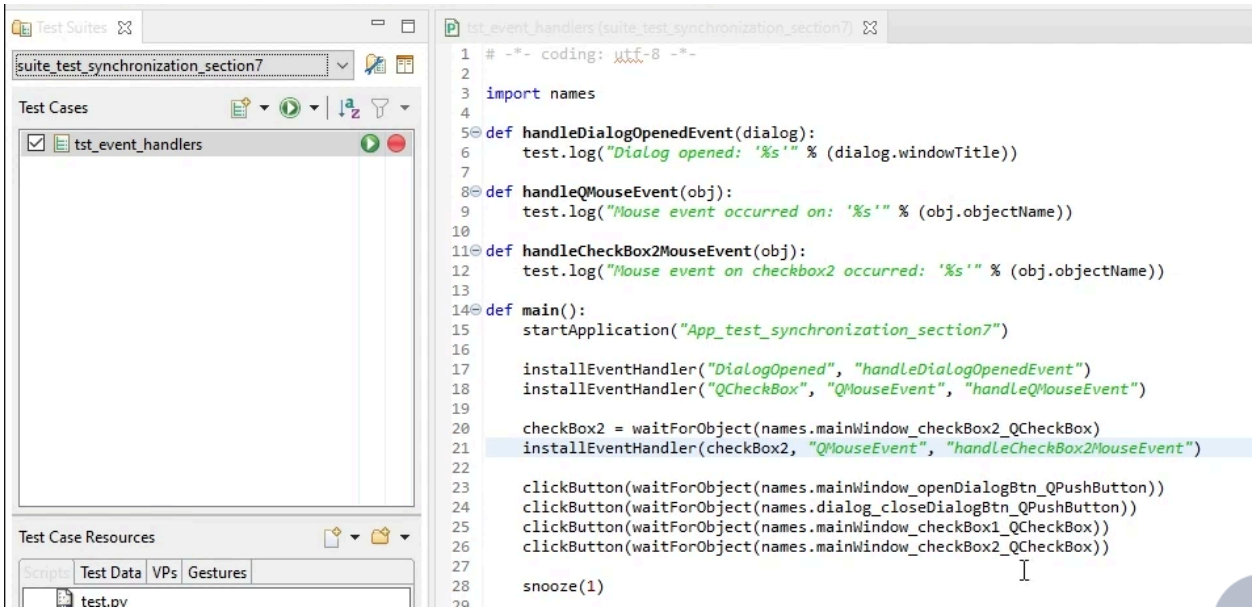
```
Python
def handleDialogOpenedEvent(dialog):
    test.log("Dialog opened: '%s'" % (dialog.windowTitle))
…
installEventHandler("DialogOpened", "handleDialogOpenedEvent")
```

It is also possible to install event handlers for specific classes and specific objects. This is done using the installEventHandler(className, eventName, handlerFunctionName) function for

specific classes and the installEventHandler(object, eventName, handlerFunctionName) function for specific objects. For these types of events, the event handler function should include an additional parameter containing a reference to the object that emitted the event. The eventName for both functions can be any standard Qt event type. The C++ event types should always be used rather than their QML counterparts; for example, QMouseEvent should be used instead of MouseEvent. Further information about the standard Qt event types can be found in the documentation on QEvent.

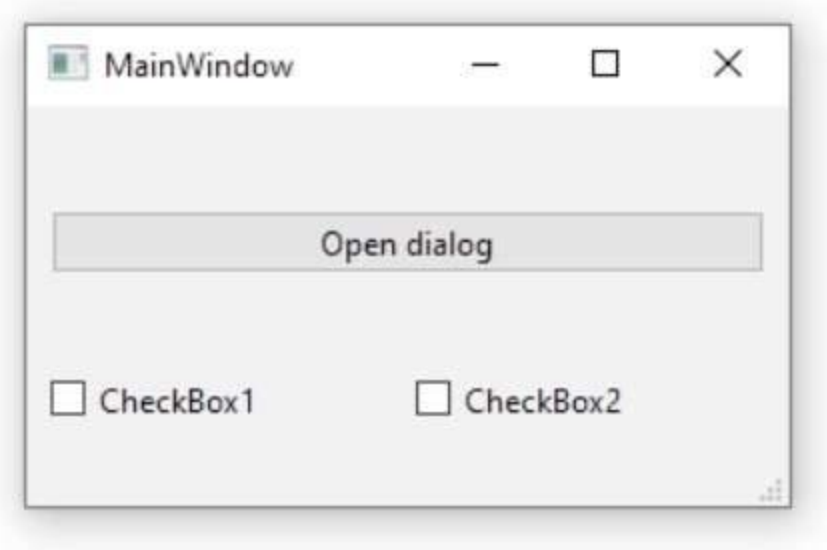
Python

```
def handleQMouseEvent(obj):
    test.log("Mouse event occurred on: '%s'" % (obj.objectName))
…
installEventHandler("QCheckBox","QMouseEvent","handleQMouseEvent")
```



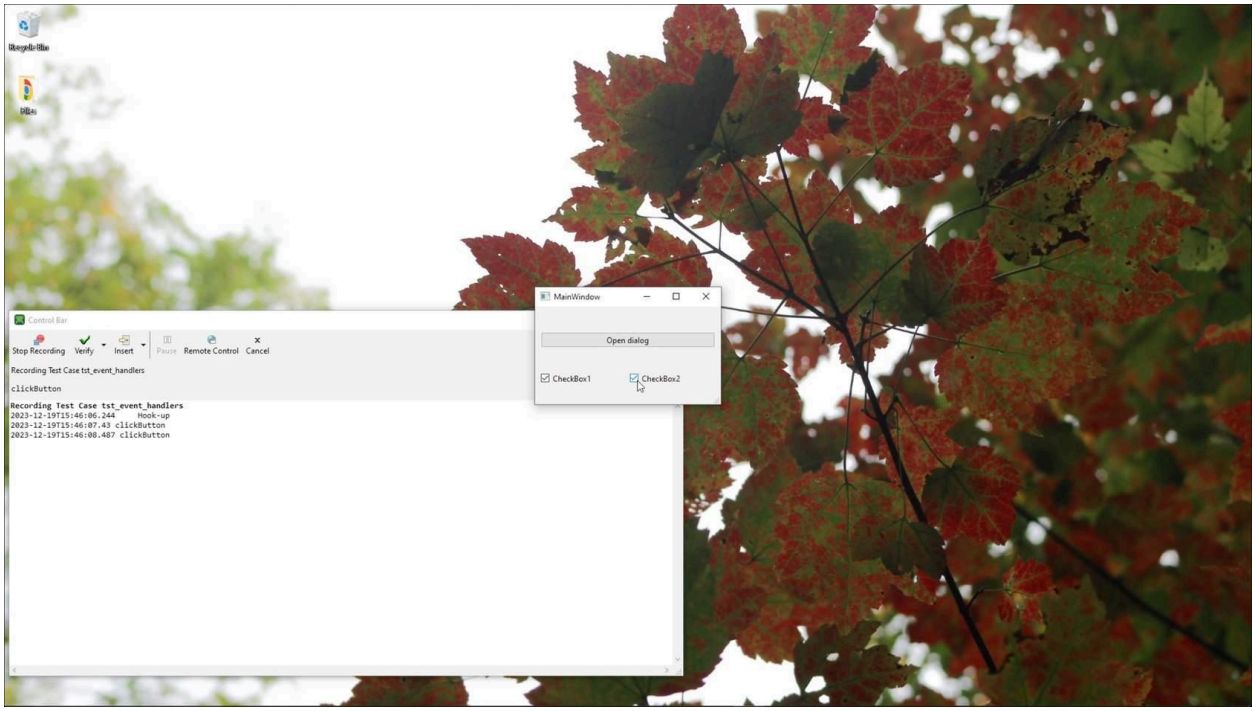
STEPS

01 Test application



For this demo, use the app located in the 'App_test_synchronization_section7' folder of the course repo. This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

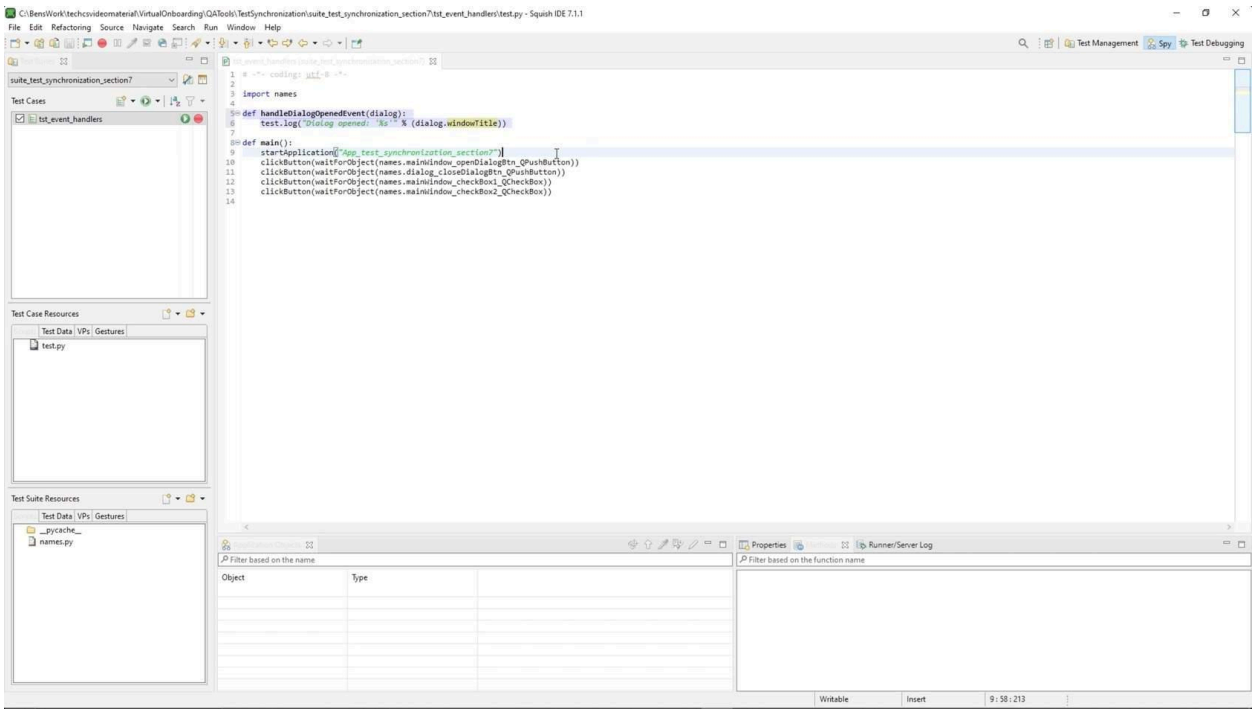
02 Record a simple test



Once setup, record a test and interact with the application's components: opening and closing a dialog and toggling checkboxes. This recorded script serves as the foundation for integrating event handlers. Stopping the recording, we will see some code has been generated in our test

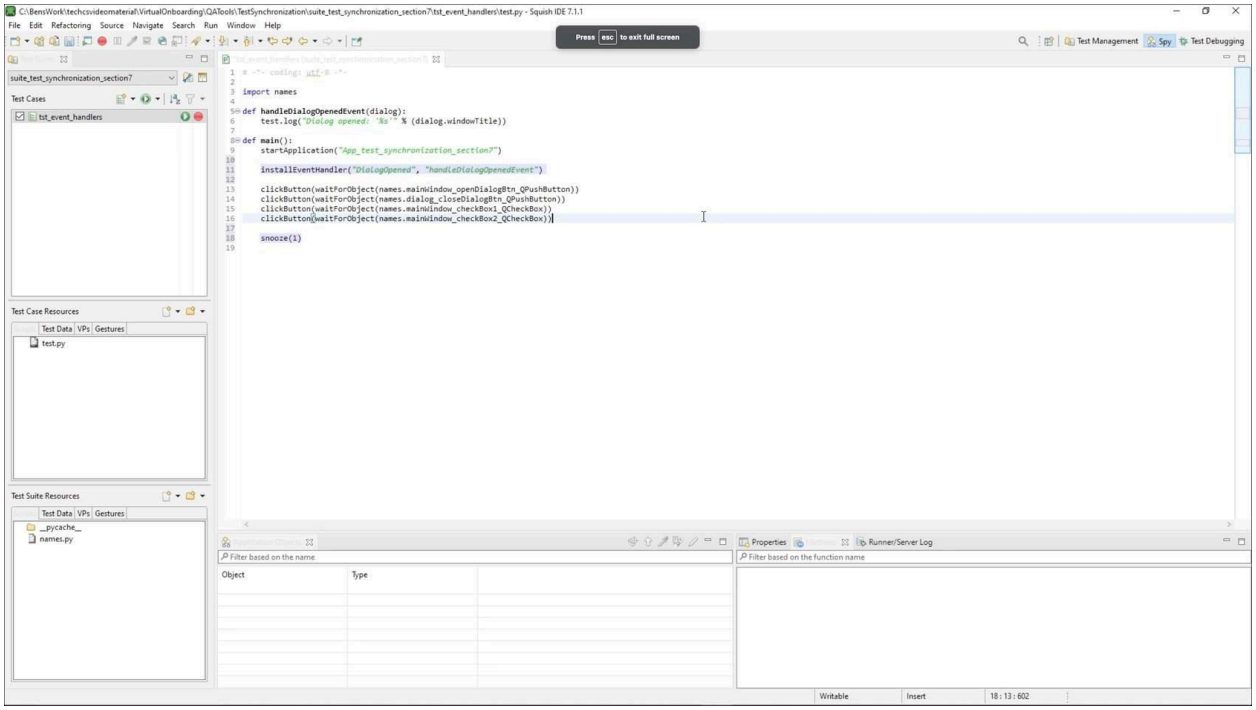
script. From here we will make some manual modifications to interact with events that occur when the AUT is running.

03 Defining an event handler



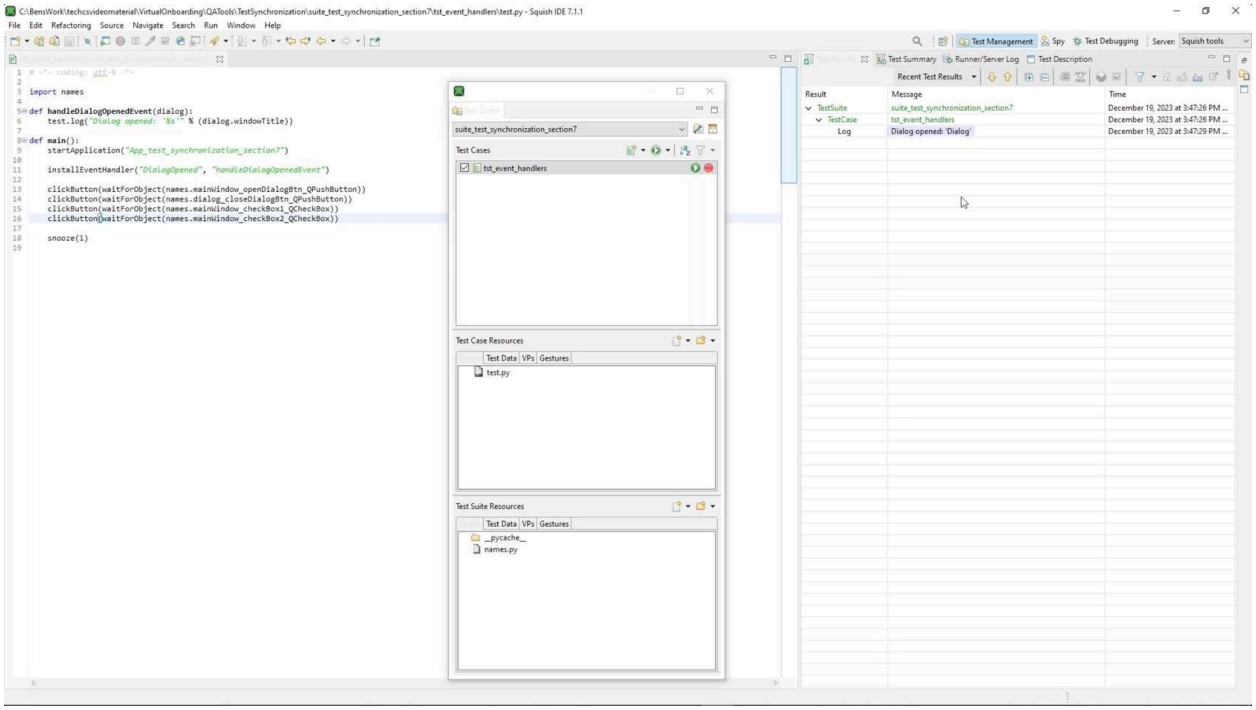
We want to be able to react to "DialogOpened", which is triggered whenever a **QDialog** is opened within the application. Let's define an event handler function called **handleDialogOpenedEvent()**, taking the dialog as a parameter.

04 Installing the event handler



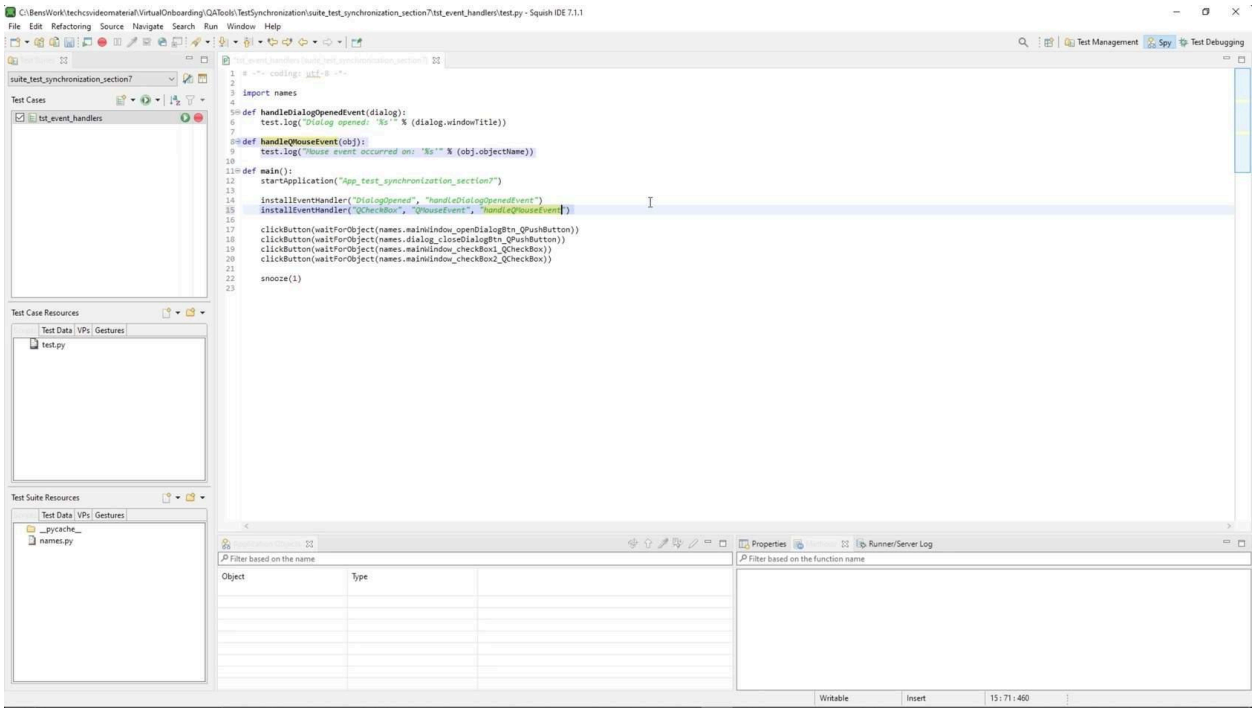
We now need to install our event handler, using the **installEventHandler()** function which takes the name of the event, and the name of the event handler as parameters. We will add this in the main test function after the application is started. We will also add a one second snooze to the end of the test to ensure that all events have time to be received.

05 Run the test



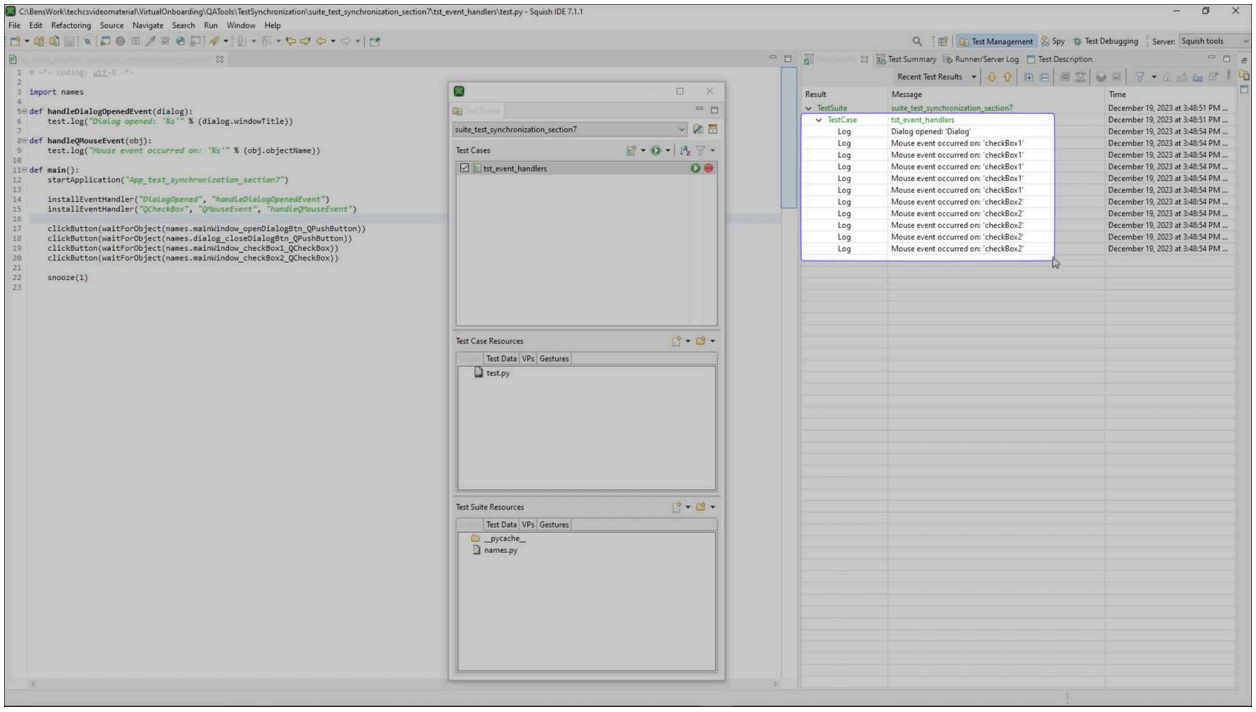
Our test should execute the **handleDialogOpenedEvent()** function anytime a **QDialog** is opened, so let us run our application to make sure this is working. The message “Dialog opened: Dialog” was logged in the test results.

06 Event handlers for component types



Define a function called a **handleQMouseEvent()** to log mouse events, passing the object as the parameter and we will log the objects name. When we install the event handler, we can provide an additional parameter which will indicate which class we would like to install it for, in this case the **QCheckBox**.

07 Run the test



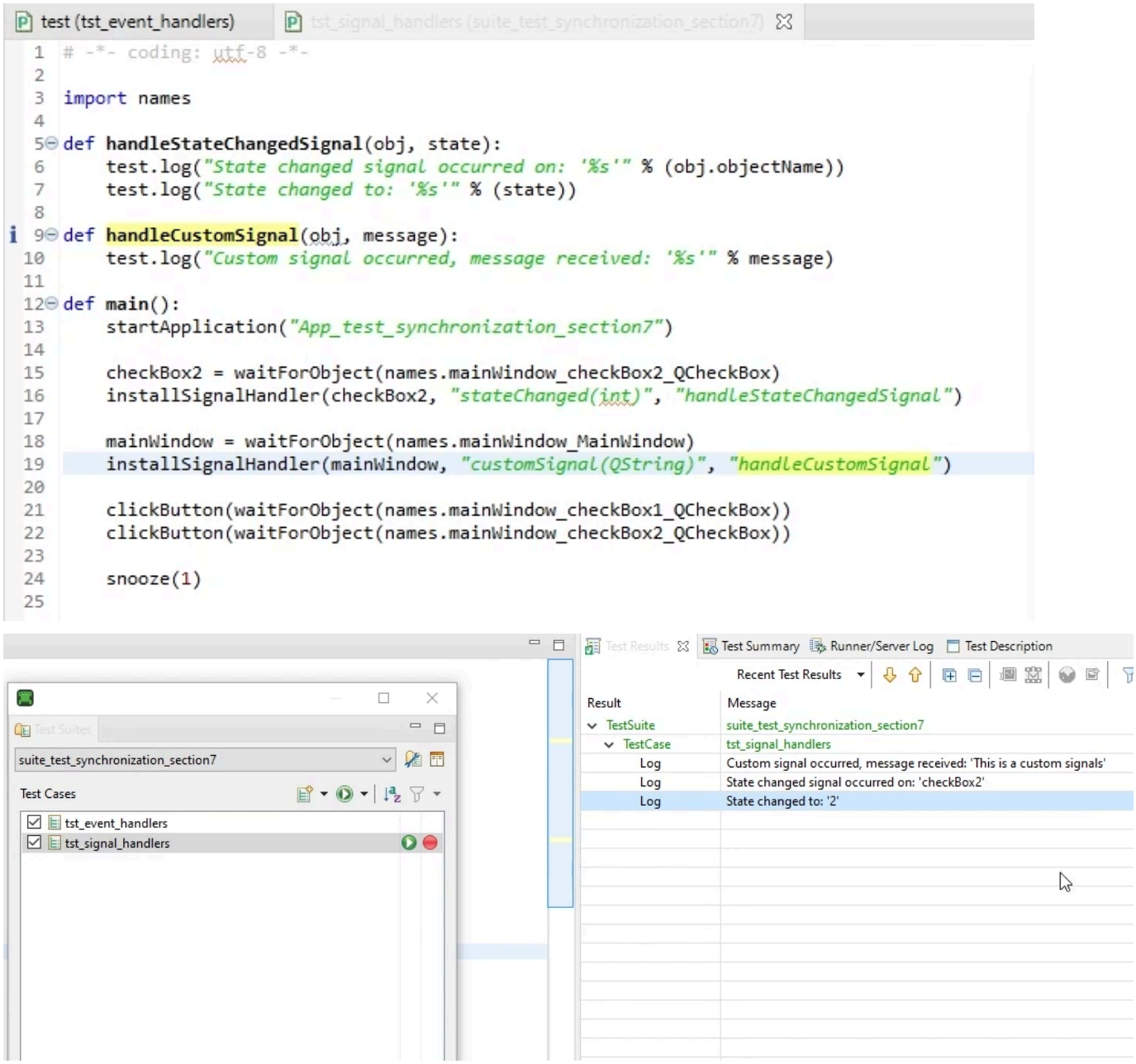
Running our test, we can see that we received mouse events for both checkbox1 and checkbox2, but no other mouse clicks. This is because we installed our event handler to target only checkboxes and no other components.

Signal Handlers

Signal handlers are functionally similar, but instead of interacting with events from the AUT, they react to Qt signals. Just like with event handers, a signal handler function must first be created outside the main test function. This function will be executed anytime the chosen signal is emitted. The parameters of the signal handler function should match that of the signal, with an additional parameter which will contain a reference to the object which emitted the signal.

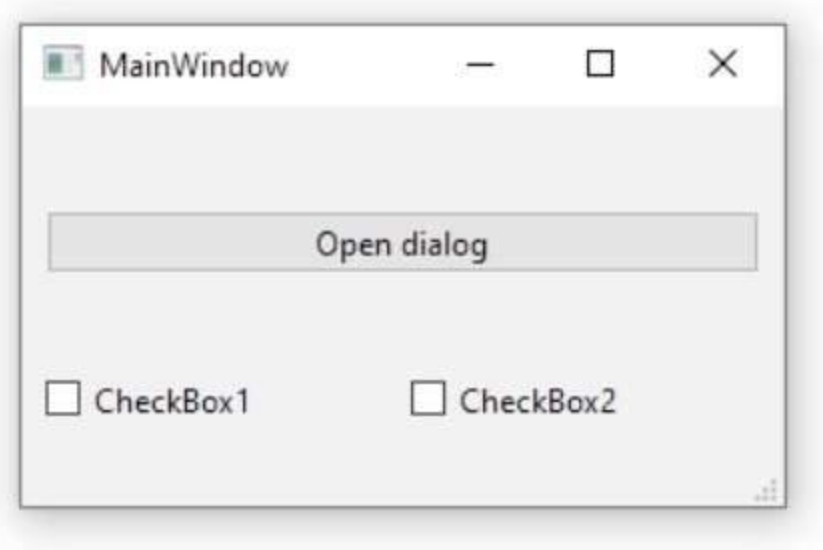
```
Python
def handleStateChangedSignal(obj, code):
    test.log("State changed signal occurred on: '%s'" % (obj.objectName))
…
checkBox2 = waitForObject(names.mainWindow_checkBox2_QCheckBox)
    installSignalHandler(checkBox2, "stateChanged(int)",
"handleStateChangedSignal")
```

Once this function has been created, the signal handler must be installed with the installSignalHandler(objectOrName,signalSignature, handlerFunctionName) function. This works similarly to the way event handlers are installed, except that a signalSignature must be provided rather than an eventName. This signalSignature must be written out as a string and must include the parameter types of the signal. The types for the signal handler should always be the C++ types rather than the QML types, for example QString should be used rather than string.



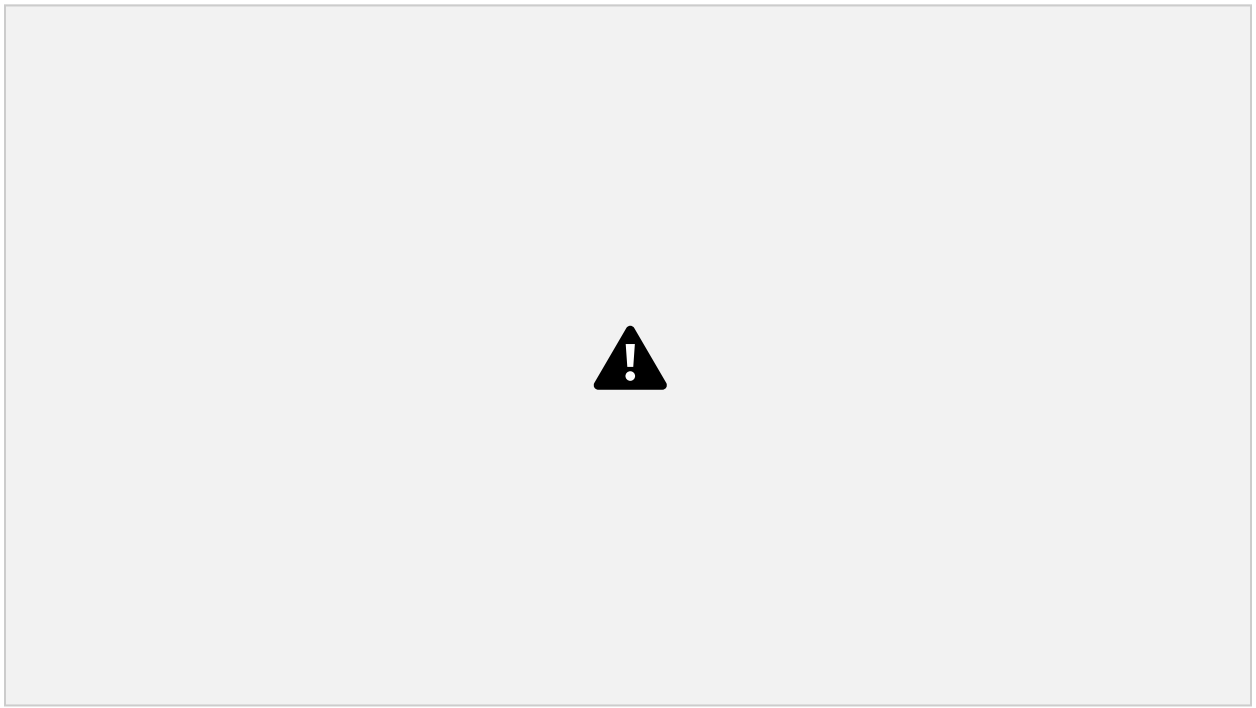
STEPS

01 Test application



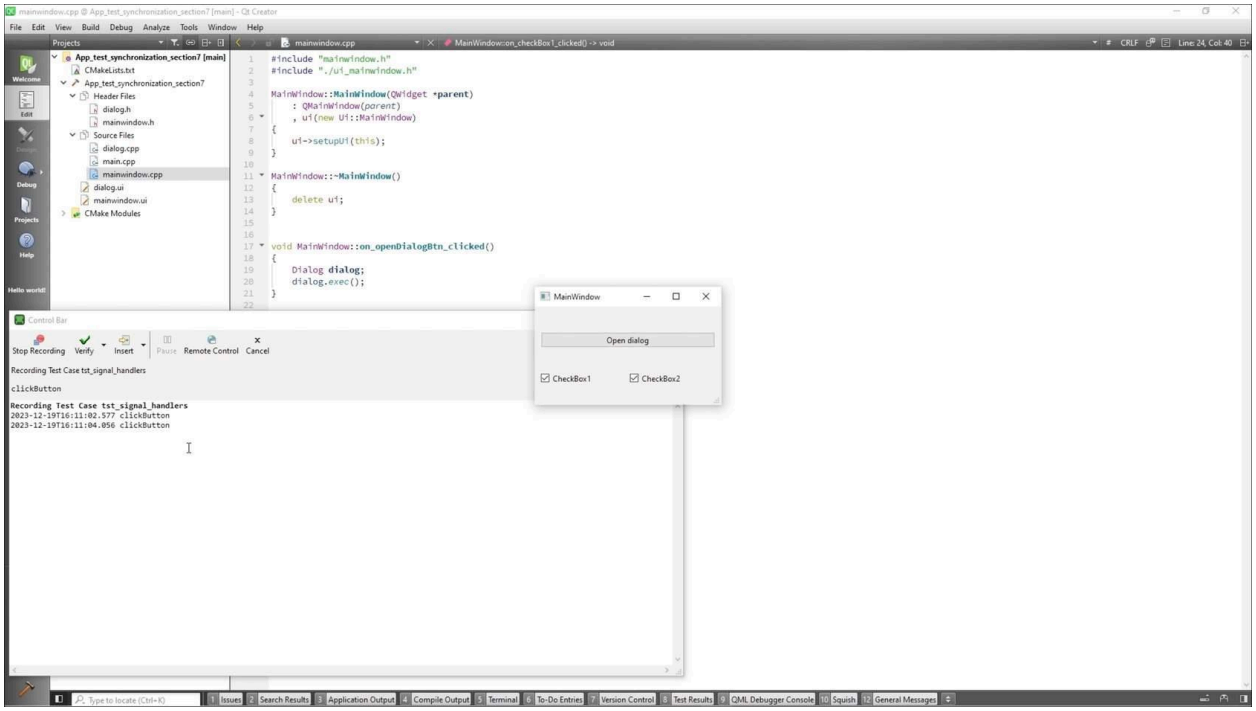
For this demo, use the app located in the 'App_test_synchronization_section7' folder of the course repo. This application was created for this demonstration and must be built for the specific Squish version and mapped in Squish to run the test scripts successfully. The application was built using Qt 6.5.2 minGW 64-bit.

02 Understanding the app



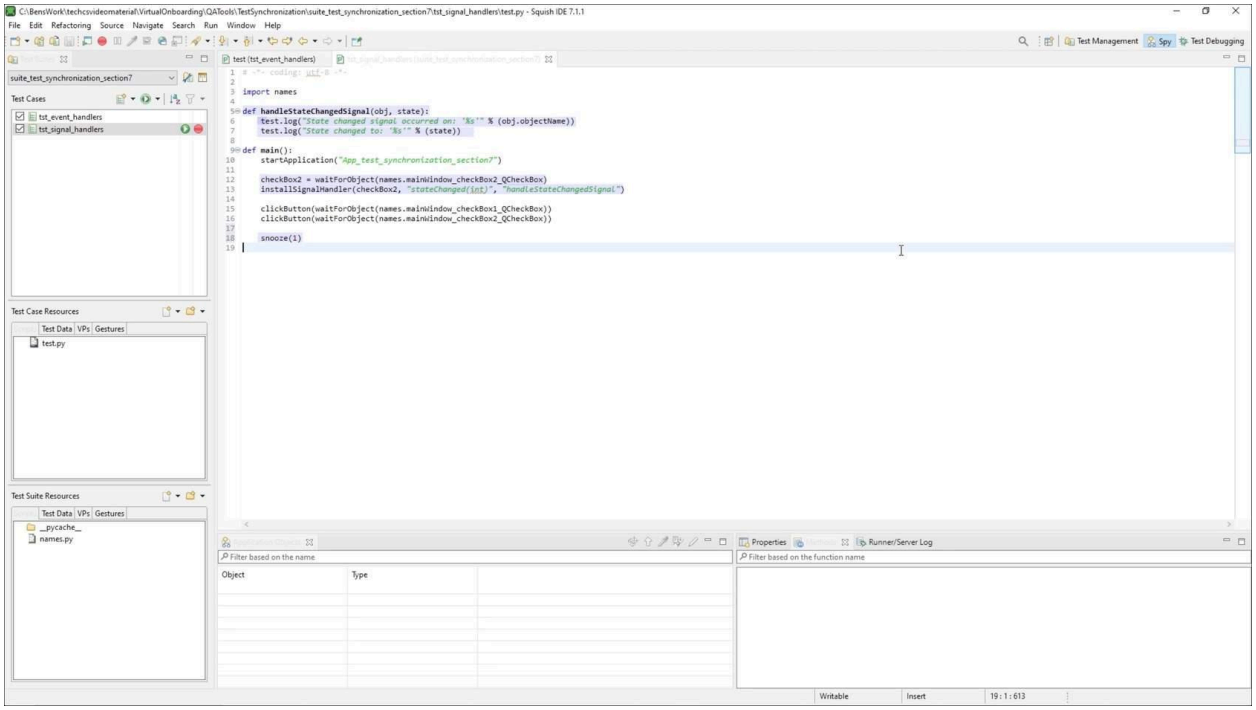
If we look at the mainwindow.cpp, we can see that checkbox 1 has a custom signal, named customSignal, emitted when checkbox1 is clicked, alongside the built-in stateChanged signal of checkboxes.

03 Record a simple test



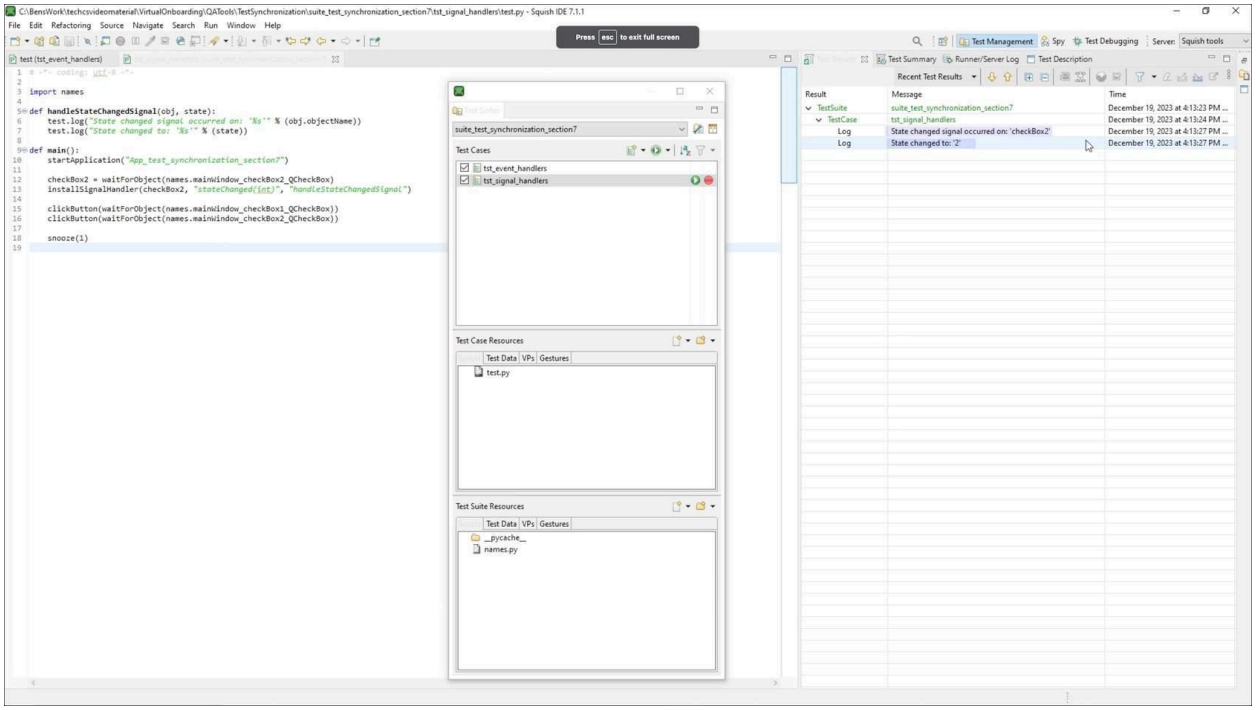
Record a test and interact with the application's components: We will simply click both check boxes, which we know will emit our custom signal, but this will also emit the state changed signal which is built into the functionality of the checkboxes, then click stop recording.

04 Handling built-in signals.



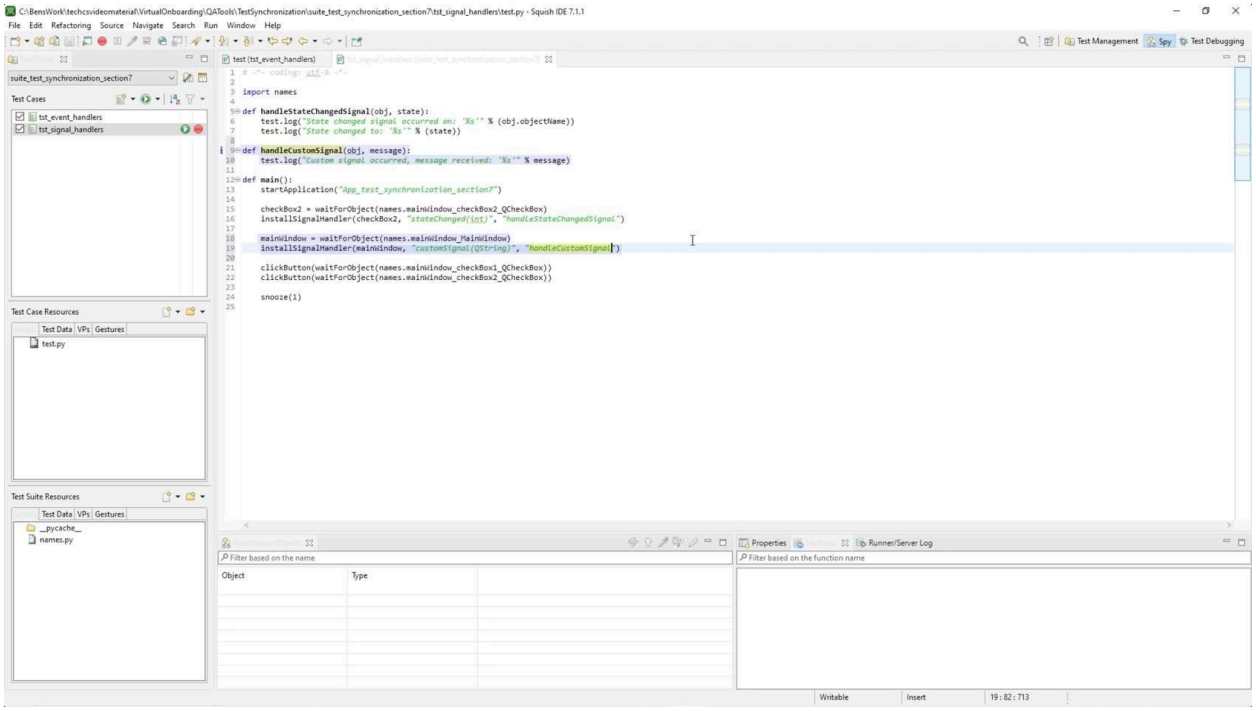
To handle **stateChanged** signal emitted by the checkboxes. We will define a function **handleStateChangedSignal**, to log the name and new state of the checkbox emitting this signal. To install the signal, the **installSignalHandler** needs 3 parameters: the object emitting the signal, the name of the signal, and the name of the signal handler function. Above **installSignalHandler,** we can retrieve and store the object. We will also add a one-second snooze to the end of the test to ensure that all signals have time to be received.

05 Run the test



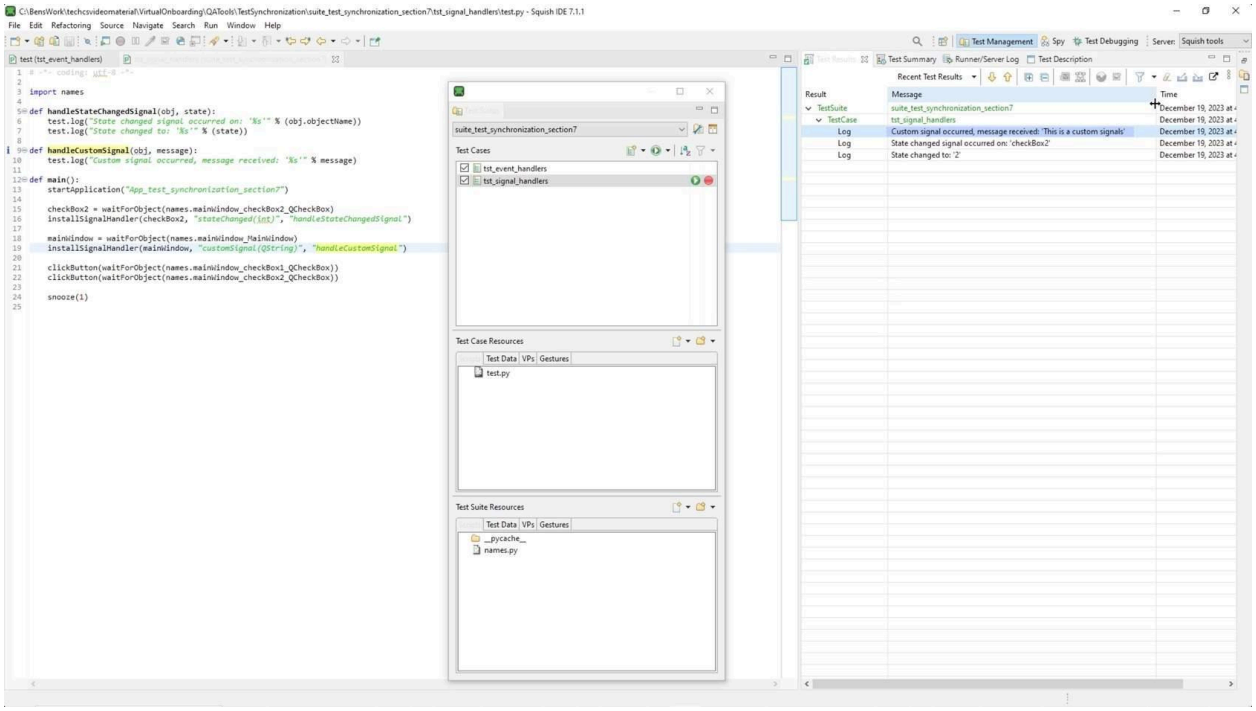
Our test should execute and we can see that we are successfully executing our signal handler function whenever the state of the second check box changes.

06 Handling custom signals



Let's add another handler, handleCustomSignal, which will have parameters for the object and a string for the message the custom signal emits. We will get the mainWindow object, store it in a mainWindow variable, and pass this to the installSignalHandler along with the custom signal and signal handler.

07 Run the test



Upon running the test, we observe successful execution of the test, both of our signal handler functions were executed by looking at the test results, and we can see the message displayed from the custom signal.

Summary

Test script and Application Under Test (AUT) should remain synchronized

When running a test script using Squish, it is important that the test script and Application Under Test (AUT) remain synchronized. This will ensure that the test script does not send commands to the AUT that the AUT is not ready to carry out.

Time-based synchronization is efficient but often sub-optimal method

Time-based synchronization in Squish refers to the ability to synchronize the test script and the Application Under Test (AUT) based on a specified amount of time. This means that the snooze(seconds) function is inserted into the test in places where the AUT may need time to run some operation, like when waiting for a window to pop up.

This method of test synchronization is often sub-optimal. This is because it is possible that the script waits shorter or longer than necessary, as there is no guarantee that conditions when the test is run will be identical to those when the test was written.

Object-based synchronization waits for an object to be available

Object-based synchronization in Squish refers to the ability to synchronize the test script and the Application Under Test (AUT) based on the presence or state of a particular object in the AUT's user interface. This means that Squish will wait for an object to “exist” before attempting to interact with it.

Object-based synchronization is almost always preferable to time-based synchronization, as Squish will wait the exact amount of time necessary before attempting to interact with an object. Because of this, object-based synchronization is the default method Squish uses when recording a test case.

Image-based synchronization waits for an image to be visible

Image-based synchronization in Squish refers to the ability to synchronize the test script and the Application Under Test (AUT) based on the presence or appearance of a particular image in the AUT's user interface. This means that the test script can wait for a specific image to be displayed in the AUT before proceeding to the next step.

Image-based synchronization is conceptually similar to object-based synchronization. However, instead of waiting for an object to be available, it waits for an image to be visible.

Text-based synchronization relies on Tesseract for text recognizition

Text-based synchronization refers to the ability to synchronize the test script and the Application Under Test (AUT) based on the text displayed in the AUT's user interface. This means that the test script can wait for a specific text to be displayed in the AUT before proceeding to the next step.

Text-based synchronization is similar to object-based synchronization and image-based synchronization. However, it relies on an external engine for text recognition, Tesseract being the recommended engine.

In conditional synchronization a synchronization point is based on a condition

Conditional synchronization refers to the ability to synchronize the test script and the Application Under Test (AUT) based on certain conditions that occur during the testing process. This means that the test script will only proceed to the next step once the specified condition has been met or an optional timeout is reached.

Test synchronization with event and signal handlers is based on specific events or signals that are emitted by the AUT

Event and signal handlers in Squish refer to the ability to synchronize the test script and the Application Under Test (AUT) based on specific events or signals that are emitted by the AUT during the testing process. This means that the test script can wait for a specific event or signal to be emitted by the AUT before proceeding to the next step.
This method relies on the usage of event handlers or signal handlers.
Event handlers interact with events, and signal handlers interact with Qt signals. In both cases, a handler function with the correct parameters must first be defined outside the main test function of the test. Once the handler function has been created, the event or signal handler must be installed.